

Discrete Event Simulation as a Predictor for Factory Traffic Management

By

Esteban Ramirez Echavarria
B.S., Mechatronics and Production Engineering
Universidad Iberoamericana, 2019

Submitted to the Department of Electrical Engineering and Computer Science and the Sloan School of Management in partial fulfillment of the requirements for the degrees of Master of Business Administration and Master of Science in Electrical Engineering and Computer Science

In conjunction with the Leaders for Global Operations Program
at the MASSACHUSETTS INSTITUTE OF TECHNOLOGY

May 2025

© 2025 Esteban Ramirez Echavarria. All rights reserved.

The author hereby grants to MIT a nonexclusive, worldwide, irrevocable, royalty-free license to exercise any and all rights under copyright, including to reproduce, preserve, distribute, and publicly display copies of the thesis, or release the thesis under an open-access license.

Authored by: Esteban Ramirez Echavarria
MIT Sloan School of Management and
Department of Electrical Engineering and Computer Science
May 9th, 2025

Certified by: Luca Daniel
Professor of Electrical Engineering and Computer Science
Thesis supervisor

Certified by: Steven Spear
Senior Lecturer at the Sloan School of Management
Thesis supervisor

Accepted by: Leslie A. Kolodziejewski
Professor of Electrical Engineering and Computer Science
Chair, Department Committee on Graduate Students

Accepted by: Maura Herson
Assistant Dean, MBA Program, MIT Sloan School of Management

Discrete Event Simulation as a Predictor for Factory Traffic Management

By Esteban Ramirez Echavarria

Submitted to the Electrical Engineering and Computer Science Department and the Sloan School of Management in partial fulfillment of the requirements for the degrees of Master of Science in Electrical Engineering and Computer Science and Master in Business Administration

ABSTRACT

Manufacturing environments increasingly rely on automation and data-driven decision-making to optimize efficiency and production rates. This study explores the application of Discrete Event Simulation (DES) to model material flow and predict AGV (Automated Guided Vehicle), crane, and cart movements within a factory. The goal is to develop a digital twin that enables real-time decision-making, optimizes scheduling, and minimizes bottlenecks.

To achieve this, we utilize SimPy, an open-source Python-based DES library, in conjunction with a custom-built API and React.js front-end interface. The study evaluates available DES software options and justifies the selection of SimPy based on flexibility, integration capabilities, and its suitability for modeling custom business rules. The solution is structured into modular components handling path planning, transporters, flows, stations, hot-cold starts, and utilities, ensuring adaptability to future improvements.

A validation framework was established, utilizing historical data comparison and real-time validation to assess the simulation's predictive accuracy. Over a 40-day testing period, the simulation achieved 89.6% accuracy and a sensitivity, or true positive rate (TPR), of 80.2%. The simulation provides a reliable first-pass scheduling tool that can be further refined with improved data collection.

The findings indicate that while full automation of AGV deployment is not yet feasible, this study lays the foundation for future integration with the factory's Vehicle Management System (VMS). Business implications include the potential for automated scheduling, enhanced material flow visibility, and optimization of capacity planning. Future work should focus on improving data accuracy, integrating live factory data streams, and refining algorithms for predictive scheduling.

Thesis supervisor: Luca Daniel

Title: Professor of Electrical Engineering and Computer Science

Thesis supervisor: Steven Spear

Title: Senior Lecturer at the Sloan School of Management

Acknowledgements

I would like to thank my colleagues at Boeing for all their support in completing this project, as well as Jason Choi for sourcing and managing the project. Additionally, I would like to thank Boeing for hosting this internship and for continuing to support the LGO program. Thank you to Luca Daniel for his technical guidance and support; without it, I could not have completed this project. Thank you as well to Steve Spear for his insightful suggestions and continued feedback.

I would like to thank MIT and the Leaders for Global Operations Program. Scott, Patty, Thomas, and Josh, thank you for taking care of your students and supporting our growth over the past two years. To my classmates, I am grateful for your friendship and support. Thank you to Arun and Yutao for sharing long writing sessions with me.

Special thanks to my partner, Ines. Thank you for bearing with me through these long two years. You have been my lifesaver when I was drowning, my rock when I needed ground and the smile I needed on tough days. Y

Finally, I would like to thank my family and friends for their outstanding support over the past couple of years. Thanks to my mom and dad, who were always willing to help. I couldn't have made it here without your support.

Table of Contents

<i>Discrete Event Simulation as a Predictor for Factory Traffic Management.....</i>	<i>1</i>
Discrete Event Simulation as a Predictor for Factory Traffic Management.....	3
<i>ABSTRACT.....</i>	<i>3</i>
<i>Acknowledgements.....</i>	<i>5</i>
<i>List of Figures.....</i>	<i>8</i>
<i>Chapter 1: Introduction.....</i>	<i>11</i>
1.1 Motivation and Problem Statement	11
1.2 Research Objectives and Scope.....	12
1.3 Significance of the Study	13
<i>Chapter 2: Background and Context.....</i>	<i>14</i>
2.1 Process Definition.....	14
2.2 Key Concepts.....	16
2.3 Organizational Context.....	20
2.3 Key Terminologies, Definitions, and Assumptions	23
<i>Chapter 3: Methods</i>	<i>31</i>
3.1 Overview of the Research Design	31
3.2 Data Collection.....	33
3.3 Tools, Frameworks, and Models.....	37
3.4 Solution Design	41
3.4.1 Path planning.....	41
3.4.2 Transporters	43
3.4.3 Flows	44
3.4.4 Stations	44
3.4.5 Hot and Cold Start	44
3.4.6 Utilities.....	45
3.4.7 API Connection	46
3.4.8 Front-end Design	47
3.5 Validation and Reliability	50
<i>Chapter 4: Results and Analysis.....</i>	<i>53</i>
4.1 Findings	53
4.3 Business Implications	56
4.4 Technical Implications and Improvements	57

4.5 Limitations of the Study	57
Chapter 5: Conclusion	58
5.1 Key Contributions	58
5.2 Recommendations and Future Work.....	59
Appendix	61
Appendix 1 – Accuracy and TPR improvements from modifying parameters.....	61
Bibliography	62

List of Figures

FIGURE 1 - BIRDS' EYE VIEW OF FACTORY LAYOUT. BOXES REPRESENT DIFFERENT STATIONS WHERE PARTS ARE PROCESSED.....	15
FIGURE 2 - SIMPLIFIED DIAGRAM OF PROCESS FLOW. ESSENTIALLY, ALL PARTS IN THE FACTORY GO THROUGH THIS SAME PROCESS, ALTHOUGH STEPS 1,2, AND 5 HAPPEN MULTIPLE TIMES. FOR EXAMPLE, A PART MAY MOVE INTO TRIM, STAGING, AND PAINT, ALL OF WHICH WOULD BE STEP 5 OPERATIONS.	16
FIGURE 3 – EXAMPLE OF SIMIO GRAPHIC USER INTERFACE. HERE, WE SEE THE GRAPHIC USER INTERFACE (GUI) AS WELL AS THE DIFFERENT DRAG-DROP COMPONENTS SUCH AS “SOURCE,” “SINK,” AND “SERVER” (IN THE LEFT PANEL). THE MIDDLE PANEL SHOWS THE COMPONENTS OF THE SYSTEM AND THEIR INTERACTIONS. ON THE RIGHT, USERS CAN ADD PARAMETERS SUCH AS PROCESSING SPEEDS. THE TOP RIBBON ALLOWS USERS TO RUN SIMULATIONS FOR A SPECIFIED AMOUNT OF TIME. THIS IMAGE ILLUSTRATES THE ADVANTAGES OF USING A DES SOFTWARE SUITE. IN CONTRAST, A DES PROGRAMMING LIBRARY IS SIMPLY A SET OF FUNCTIONS AND CONFIGURATION FOR BUILDING SIMULATIONS WITH CODE FORM SCRATCH.	18
FIGURE 4 - FACTORY LAYOUT AS SEEN FROM A BIRDS-EYE VIEW. THE “CRUCIBLE” STATION, WHICH RECEIVES PARTS OF ALPHA, BETA, AND GAMMA, IS HIGHLIGHTED IN RED. EACH GRAY BLOCK REPRESENTS A STATION. THE STATIONS ON THE TOP HALF OF THIS PICTURE PERFORM POST-CURE OPERATIONS, WHEREAS THE STATIONS AT THE BOTTOM HALF OF THE PICTURE REPRESENT PRE-CURE OPERATIONS. THE RED MARKERS ON THIS IMAGE REPRESENT “NODES” IN HALLWAYS THROUGH WHICH TRANSPORTERS MOVE TO TRANSFER PARTS FROM ONE STATION TO THE NEXT. AS THE CRUCIBLE IS REQUIRED FOR THE PRODUCTION OF ALL PART TYPES, HANDLING TRAFFIC IN THE NODES ADJACENT TO IT HAS PRESENTED A BIG CHALLENGE. THE HORIZONTAL HALLWAY IN THE MIDDLE OF THE IMAGE IS NOT USED FOR TRANSPORTATION.	22
FIGURE 5 - ALPHA, BETA, AND GAMMA PART PROCESS LOCATIONS. THIS PICTURE COMPLEMENTS FIGURE 4 BY DEPICTING THE GENERAL LOCATIONS FOR THE WORK CENTERS THAT HANDLE ALPHA, BETA, AND GAMMA PARTS. THE SECTION HIGHLIGHTED IN GREEN REPRESENTS STATIONS PRIMARILY FOCUSED ON THE PRODUCTION OF ALPHA PARTS. AS SUCH, TRAFFIC IN THESE AREAS WILL ONLY AFFECT THE PRODUCTION OF ALPHA PARTS. THE SAME IS TRUE FOR THE STATIONS HIGHLIGHTED IN RED AND BETA PARTS. THE FACTORY SECTION HIGHLIGHTED IN BLUE WILL PERCEIVE THE MOST TRAFFIC DUE TO TWO MAIN REASONS: THE CRUCIBLE STATION IS LOCATED HERE, AND GAMMA PARTS REPRESENT THE LARGEST VOLUME OF PRODUCTION.	22
FIGURE 6 - DIFFERENT CONFIGURATIONS FOR GAMMA PARTS DURING THE PRE-CURE PROCESS. THIS DIAGRAM IS A VISUAL REPRESENTATION OF HOW PARTS GAMMA PARTS CAN BE CONFIGURED ON GAMMA-A FIRST STATION. ON ATLAS TOOLS OF ESSENTIALLY THE SAME SIZE, EITHER 2, 4, OR 6 GAMMA PARTS CAN BE PLACED.	27
FIGURE 7 - GAMMA SPLITTING PROCESS. PRE-CURE PARTS ARE MOUNTED IN AN ATLAS TOOL. THEY'RE SPLIT INTO CARTS AND, FINALLY, PLACED INTO INVENTORY. FINALLY, WHEN READY TO MOVE, THEY'RE PLACED AMONG OTHER GAMMA PARTS ON CASSETTES.	28
FIGURE 8 - SCREENSHOT OF SIMULATION SYSTEM FRONT-END APPLICATION. THIS SCREENSHOT SHOWS A BIRDS-EYE VIEW OF THE FACTORY FLOOR'S CURRENT STATE. THE GREEN BARS ON EACH STATION INDICATE THE PROGRESS LEVEL OF A PART IN THAT STATION. YELLOW STATIONS INDICATE PARTS THAT HAVE BEEN COMPLETED AND ARE WAITING TO BE MOVED. THE LEFT PANEL SHOWS ADDITIONAL DETAILS OF THE CONTENTS OF A GIVEN STATION. THE TOP SECTION CONTROLS ALLOW USERS TO RUN AND VISUALIZE THE SIMULATION FOR A TOTAL OF 10 HOURS. OTHER COMPONENTS SUCH AS CRANES, AGVS, CARTS, AND TOOLS CAN BE EASILY INSPECTED IN THE SIMULATION. STATION NAMES HAVE BEEN REDACTED WITH A GRAY BLOCK.	32
FIGURE 9 - EXAMPLE OF STAMP PROCESSING TIMES AS A PERCENTAGE OF TOTAL OPERATION PROCESSING TIME. THIS EXAMPLE SERVES AS A VISUAL EXPLANATION OF STAMP PROCESSING TIMES, WHERE SOME STAMPS	

(LIKE STAMP F) MAY TAKE UP A SIGNIFICANT AMOUNT OF TIME TO PROCESS, IN CONTRAST TO OTHER STAMPS, SUCH AS STAMP C.	35
FIGURE 10 - VISUAL REPRESENTATION OF THE DIFFERENT LOGICAL LEVELS OF DATA. AS AN EXAMPLE, ONE ALPHA PART, REPRESENTED IN ORANGE, HAS SEVERAL “ORDERS” ASSOCIATED WITH IT. EACH ONE OF THESE ORDERS HAS SEVERAL OPERATIONS, AND EACH OPERATION HAS SEVERAL STEPS (OR STAMPS). THIS DIAGRAM IS A VISUAL REPRESENTATION OF HOW THE DATA IS STRUCTURED AND USED TO CALCULATE A PART’S PROGRESS.	37
FIGURE 11 - COMPREHENSIVE COMPARISON OF SIMULATION SOFTWARE. THIS TABLE REPRESENTS THE COMPARISON USED TO DETERMINE WHICH SIMULATION SOFTWARE WAS BEST SUITED FOR OUR APPLICATION.	38
FIGURE 12 - THIS SCREENSHOT SHOWS THE SETUP OF THE CWC PROCESS IN SIMIO’S SIMULATION INTERFACE. THE DEVELOPMENT WAS NOT COMPLETED, BUT THIS SCREENSHOT SHOWS THE COMPLEXITY OF THE SIMULATION.	39
FIGURE 13 - THIS DIAGRAM DEMONSTRATES HOW NODES ARE REPRESENTED ON THE FACTORY FLOOR. IN THIS BIRDSEYE VIEW DIAGRAM, STATIONS ARE REPRESENTED AS ORANGE BLOCKS, AND POSSIBLE PATHS THROUGH NODES ARE BLUE DOTS.	42
FIGURE 14 - PATH PLANNING ALGORITHM. DIJSTRA’S ALGORITHM IS A TRADITIONALLY USED SHORTEST-PATH ALGORITHM, WHICH WE APPLY DIRECTLY HERE. THIS ALGORITHM SHOWS HOW A MASK IS APPLIED TO OBTAIN THE AVAILABLE NODES AND THEN THEN THE SHORTEST-PATH IS COMPUTED.	43
FIGURE 15 - PUSH-OUT ALGORITHM. THIS ALGORITHM IS USED TO EVALUATE WHETHER A PART CAN MOVE INTO THE NEXT STATION OR NOT. IF THE STATION HAS CAPACITY, THE PART CAN MOVE IN DIRECTLY. IF THE STATION IS FULL, WE CHECK IF THERE’S AN IDLE PART AT THE STATION, AND WE CHECK IF THE IDLE PART CAN BE MOVED TO A BUFFER OR STORAGE AREA. IF THAT’S THE CASE, WE MOVE THE PART TO THAT STATION. THIS ALGORITHM IS IMPORTANT FOR PUSH STATIONS.	46
FIGURE 16 - SCREENSHOT OF UX. THIS PANEL SHOWS THE UX FOR MOVING OR EDITING A PART. IN AN IDEAL CASE, PARTS’ LOCATIONS ARE AUTOMATICALLY EXTRACTED, YET THIS IS NOT ALWAYS THE CASE. THIS UX WAS DESIGNED TO MOVE INCORRECTLY PLACED PARTS, OR ADD PARTS THAT ARE NOT SHOWN IN THE SYSTEM.	48
FIGURE 17 - SCREENSHOT OF MOVE LOG. SIMULATIONS IN OUR PROPOSED SOLUTION RUN FOR 10 HOURS AND GENERATE A LIST OF MOVES WITH TIMES AND LOCATIONS. THE LIST OF MOVES IS DISPLAYED IN THIS PANEL FOR EASY EXTRACTION AND COMPARISON WITH GROUND TRUTH, AS WELL AS FOR DAY-TO-DAY APPLICATIONS.	49
FIGURE 18 - SCREENSHOT OF INSTRUCTION MANUAL PANEL. THE SYSTEM HAS SEVERAL COMPONENTS AND A DECENT AMOUNT OF COMPLEXITY. THIS INSTRUCTION PANEL WAS DESIGNED TO PROVIDE USERS WITH AN OVERVIEW OF THE FUNCTIONALITY AND COLOR CODES OF THE SIMULATION.	49
FIGURE 19 – SIMULATION ACCURACY IMPROVEMENTS USING HISTORIC DATA OVER 30 TIME PERIODS. A TOTAL OF 22 EXPERIMENTS WERE RUN, PERFORMING CORRECTIVE ACTIONS AND TUNING THE SIMULATION TO BETTER APPROXIMATE REALITY. MORE INFORMATION ON THE ACTIONS PERFORMED FOR EACH NEW EXPERIMENT CAN BE FOUND IN APPENDIX 1.	52
FIGURE 20 - CONFUSION MATRIX RESULT OF LAST SIMULATION RUN. THIS TABLE SHOWS THE SIMULATED RESULTS AS COMPARED WITH GROUND TRUTH FROM THE GATHERED DATA.	53
FIGURE 21 - REAL VS PREDICTED VARIABILITY. THIS CHART SHOWS HOW OUR PREDICTIONS PERFORM AGAINST REALITY FOR EVERY STATION. ESSENTIALLY, THE CLOSER A BAR IS TO ZERO, THE BETTER IT’S PERFORMING. AT FIRST GLANCE, IT SEEMS THAT PROCESSING TIMES FOR SOME STATIONS SHOULD BE CORRECTED, YET MODIFYING THOSE VALUES DRASTICALLY REDUCES ACCURACY. THE MAXIMUM ACCURACY WAS OBTAINED WITH THE VARIABILITY SHOWN IN THIS FIGURE.	54
FIGURE 22 - ACCURACY AND TPR MEASURED ON SIMULATIONS RUNNING OVER AN INCREASING NUMBER OF DAYS. THIS CHART INDICATES HOW INCREASING THE NUMBER OF DAYS AGAINST WHICH OUR SIMULATION IS RUNNING IMPROVES TPR, SO IF WE RUN THE SIMULATION FOR 5 DAYS AND COMPARE THE LOCATIONS OF PARTS, THERE’S A HIGHER MATCH THAN WHEN WE ANALYZE ONLY ONE DAY.	55

Chapter 1: Introduction

1.1 Motivation and Problem Statement

The 777X airplanes are the newest members of Boeing's widebody family. This project studies the manufacturing process solely at the Composite Wing Center (CWC), where carbon fiber wings are produced. As industrial automation becomes more widely adopted, certain challenges, such as managing traffic for automated vehicle movement, have emerged in advanced factories utilizing these technologies.

Factories and warehouses employing Automated Guided Vehicles – hereinafter referred to as AGVs – face the challenge of configuring AGVs in an integrated environment with a vehicle management system – hereinafter referred to as VMS – to ensure the AGVs operate in unison.

The factory requires the use of AGVs to move material, but these movements are not automated and require operator remote-control input, failing to take advantage of the full capabilities of AGVs¹. Efforts are in place to achieve semi-automation on the factory floor². Automation requires the implementation of a scheduling system³, granting the VMS the ability to know exactly when to deploy an AGV, and where it should go. Scheduling moves requires understanding the end-to-end process in detail. Additional advantages, such as traffic efficiency improvements and automated planning, can be appreciated by the introduction of a well-designed scheduling system.

Predicting traffic efficiently requires understanding the complexity of the system. Nonetheless, taking a modular approach to developing the simulation allows the mitigation of stochastic variability. Several simplifying assumptions must be made to scope a DES project that solves the

¹ A properly implemented AGV setup should work autonomously without human intervention. AGVs in the factory follow magnetic “dots” to navigate around the factory, and a magnetic line to move into a cell. The dot navigation currently is ignored, and instead operators use a remote control to move the AGVs.

² There are 6 levels of autonomous vehicles, for simplicity, in this paper we consider only 3: No automation, semi-automation, and full automation. These correspond to levels 0, 3, and 5 of the Society of Automotive Engineers (SAE) levels of automation. Source: SAE Levels of Driving Automation, May 3rd, 2021.

³ A learning that emerged after evaluating the system’s complexity.

task at hand. Untangling the system's dependencies enables a modular approach to traffic management⁴.

Process visibility can improve production by developing new mechanisms, analyzing bottlenecks, and assessing investment opportunities. The complexity of the production process can make bottlenecks particularly hard to identify, especially when the system is evaluated in cells rather than as a whole. Understanding the bottom-line impact of equipment and layout investments on production is a significant impetus for this paper.

Another motivation for this work is the need to implement alternative solutions for problems that lack data. Rule-based decision-making is a traditional approach to problems that has now been resolved more effectively by machine learning algorithms; however, these models become ineffective when substantial amounts of clean, usable data are unavailable. In low-data environments, rule-based heuristics can serve as an initial decision-making framework where predictive models falter. This paper explores how DES can complement this approach by enabling simulation-based scenario testing to bridge the data gap.

This paper aims to persuade the reader that discrete event simulation, hereafter referred to as DES⁵, is an effective tool for predicting AGV movements in a factory setting. By integrating DES into traffic management and scheduling, this study addresses the current gap in data-driven decision-making for AGV coordination. Through simulation-based analysis, we offer insights into optimizing flow efficiency, minimizing congestion, and enhancing scheduling strategies in both partially and fully automated environments.

1.2 Research Objectives and Scope

This project aims to develop a solution to the current problems of scheduling and visibility. As such, this paper has two primary objectives: to provide visibility of factory status and to enable time-based predictions using simulation-based software, namely, DES.

⁴ As explained in Steven Spear's "Wiring the Winning Organization"

⁵ Further explanation of DES in chapter 2

The final solution should be a ready-to-use interface that allows engineers and operators to better interact with and understand both the current state and future states of the factory in terms of material flow. Our simulation will be considered accurate if it predicts material flow with at least 85% accuracy . accuracy is calculated based on a confusion matrix with the following shape:

		Real	
		Moved	Didn't move
Predicted	Moved	True Positive	False Positive
	Didn't move	False Negative	True negative

Where accuracy is calculated as:

$$Accuracy = \frac{True\ Positive + True\ Negative}{True\ Positive + True\ Negative + False\ Positive + False\ Negative}$$

Accuracy will be measured against a range of dates of historical data. This study focuses on the production facility for Composite Wings. All materials, tools, and factory layout are accurate for the dates of June-December 2024. Due to the frequently changing nature of production facilities, a secondary objective for this project is designing a software system of flexible and modular⁶ for future expansion.

1.3 Significance of the Study

Automated machinery and robotics represent large investments for production facilities, yet they are often not properly implemented, and their full potential is not harnessed. Automated Guided Vehicles (AGVs) have been adopted by over 60% of production facilities,⁷ primarily for material flow. The methods and results in this research paper are transferable knowledge to

⁶ Implementing the use of “Clean Code,” and SOLID principles. See Chapter 2.1 for additional details

⁷ According to United Robotics Group article “Embracing the Next Generation of Manufacturing Automation”

companies that wish to understand how to address material flow planning in an automated environment. Moreover, this study also addresses the importance of running simulations of factory settings for optimization, measurability, and design.

Where Machine Learning (ML) and other Data Science predictive methods require large amounts of data in order to make accurate predictions, less data-intensive methods, such as Discrete Event Simulation (DES), provide a significant advantage in transforming environments such as manufacturing. As an example, introducing new machinery to a manufacturing line alters the rates and bottlenecks of a system, rendering prior data-driven models unusable, whereas DES models can be easily modified to accommodate the new environment.

With the growing adoption of Digital twins for real-time visualization, scenario testing, and data-driven decision-making, understanding the applications of DES in material flow analysis has never been more important. This study presents a real example of using simulation to optimize traffic flow and predicting congestion points before they occur. Additionally, these simulations offer a measurable and reliable method for optimizing flows and understanding bottlenecks.

Chapter 2: Background and Context

2.1 Process Definition

Boeing's 777 models are some of the company's largest. To make these airplanes more fuel-efficient, the newer models, 777X, utilize carbon fiber (composite) wings. Fabrication of these wings happens at the Composite Wing Center (CWC), where this project took place.

As composite parts cannot be welded together, they are produced spanning the whole length of the wing. Moving these parts around the factory from one station to the next requires the use of a whole aisle, causing potential traffic jams. Consider Figure 1. When a

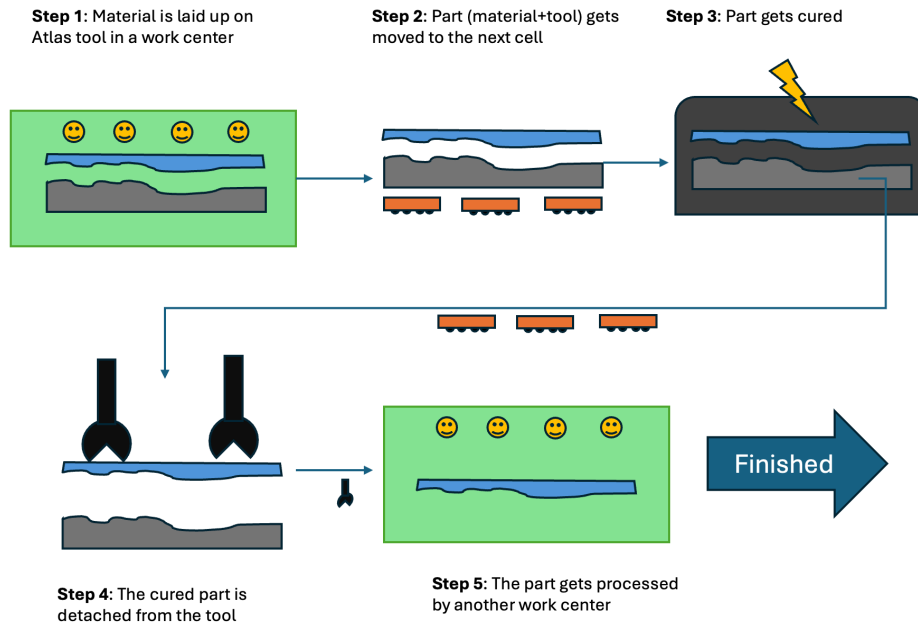


Figure 2 - Simplified Diagram of process flow. Essentially, all parts in the factory go through this same process, although steps 1, 2, and 5 happen multiple times. For example, a part may move into trim, staging, and paint, all of which would be step 5 operations.

2.2 Key Concepts

Understanding the implications and advantages of this paper requires a high-level comprehension of the historical background of industrial engineering and widely adopted software frameworks. Being DES the primary focus of this research, it's important to understand applications of simulation in factory settings, from Industry 4.0 to the different software suites available. Additionally, developing a flexible software product requires discussing the paradigms most widely adopted when developing software at scale.

Industry 4.0⁸—Expanding on the ideas of Industry 3.0, which included robotics, automation, and the introduction of computing in the manufacturing world, Industry 4.0, often referred to as the fourth industrial revolution, uses big data analytics, machine learning, Internet of Things (IoT),

⁸ Definition extracted from “Industry 4.0: critical investigations and synthesis of key findings” by Moustafa Elnadi et al.

and cloud computing to improve industrial operations. This study evaluates a novel application of data-driven decision-making in Industry 4.0-enabled factories. As market reports predict a compound annual growth rate (CAGR) of ~20% from 2024-2032⁹, the applications of DES and Digital Twins or factory material flow analysis will likely become increasingly necessary.

Digital Twin – The term “digital twin” refers to a digital model of a physical object or system¹⁰. The development of digital twins in manufacturing has gained momentum in recent years, particularly with the advent of Industry 4.0. By harnessing the power of advanced sensors and cloud computing, creating a digital replica of a physical system is an achievable goal in this era. Digital twins, by definition, should mimic the performance of their physical counterparts in real time, thus requiring live data updates¹¹. By providing a holistic view of the factory’s state, Digital Twins allow their users to visualize potential risk factors and bottlenecks in their manufacturing line. It’s crucial to emphasize that the successful implementation of a digital twin necessitates an in-depth understanding of the system to be analyzed and accurate up-to-date measurements of the full state of the system.

Discrete Event Simulation (DES) – While Digital Twins provide visibility of the current state of a system, simulations allow users to conduct experiments, identify risks, and, as proven by this work, make predictions of future states of the system. The term “discrete” refers to the simulation's behavior in time. The system is evaluated in time jumps defined by a list of “events”, and at each time jump, the system’s state is re-computed¹². DES is widely adopted as the standard simulation technique in the fields of logistics, manufacturing, supply chain, and healthcare. Although simple DES systems could be computed by hand, software DES allows extremely complex systems to be modeled, and enables the implementation of parallel events through the use of an internal event loop. This internal event loop allows for simulations of months or years of data in a few seconds, as well as parallelizing hundreds of events without the

⁹ As published in Globe Newswire’s “Industry 4.0 Market Size & Growth Analysis”

¹⁰ In depth review of digital twins can be found in Cyngn’s article “Applications of Digital Twins in Manufacturing”

¹¹ The subtle differences between digital twins and simulation are explored further in Cyngn’s article

¹² More information on how DES systems are constructed can be found in the book “Simulation Modeling and Analysis”

need of threaded processors. DES Software also enables stochastic modeling and resource configuration, which are useful tools for system analysis.

DES Software – In this paper we discuss two main categories of DES software: Software suites and open-source libraries. Software suites are usually licensed, have a user interface, and simplify many common tasks that users may require. Open-source libraries, on the other hand, are mainly targeted towards the programming savvy. The leading software suites include AnyLogic, Simio, FlexSim, Arena, SIMUL8, Simulink, and Plant Simulation. Some commonly used open-source libraries include SimPy, PowerDEVS, Theta, and DESMO-J used for integrating with codebases written in Python, C++, Rust, or Java, respectively. Open-source libraries offer a substantial advantage in connectivity and efficiency. Chapter 3 will detail a comprehensive exploration of these tools for the problem in this study and the rationale for choosing one against the other.

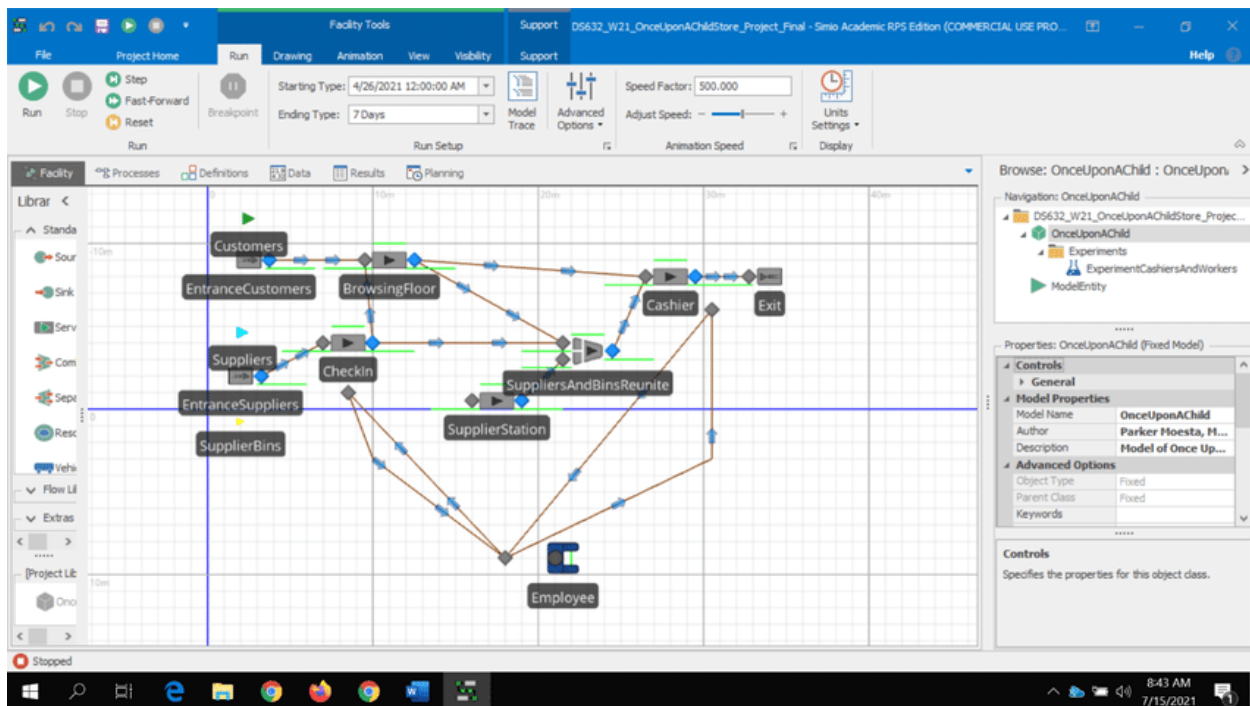


Figure 3 – Example of SIMIO graphic user interface. Here, we see the Graphic User Interface (GUI) as well as the different drag-drop components such as “Source,” “Sink,” and “Server” (in the left panel). The middle panel shows the components of the system and their interactions. On the right, users can add parameters such as processing speeds. The top ribbon allows users to run simulations for a specified amount of time. This image illustrates the advantages of using a DES Software Suite. In contrast, a DES programming library is simply a set of functions and configuration for building simulations with code from scratch.

SOLID Principles—First introduced in the book Clean Code by renowned software author and engineer Robert C. Martin (Uncle Bob), these principles are guidelines for writing maintainable, readable, and extensible code. The five principles are **S**ingle Responsibility, **O**pen-Closed, **L**iskov Substitution, **I**nterface Segregation, and **D**ependency Inversion. Understanding each is not necessary for the purpose of this paper, but applying SOLID principles in this study ensures that the simulation models remain modular and adaptable for future factory optimizations.¹³

Waterfall Model – Introduced in the 1970s¹⁴, this was the first model for structured software development. The general idea consists of following a linear set of phases from defining the requirements to implementation, testing, and finally, deployment. This model has proven to be inflexible as it requires large amounts of rework if issues are detected late in the process.

Agile Model – Developed in the 1990s as a response to the Waterfall model, the Agile Manifesto – created by 17 reputable software developers, including the aforementioned “Uncle Bob” – suggested an iterative, incremental approach with a short feedback loop. Inspired by Lean manufacturing, the core principles of the Agile Manifesto emphasize team collaboration and continuous improvement over rigid rules and extensive documentation.¹⁵

Scrum – The most widely adopted Agile-based methodology consists of time-boxed events called “sprints,” which last between two to four weeks. These development cycles enable fast iteration, review, and redirection. Scrum introduces additional roles, such as Product Owner and Scrum Master, which may be troublesome for small teams but provide value for larger teams. Implementing Scrum in this paper ensures early error correction and the satisfaction of the relevant stakeholders.¹⁶

Human Machine Interaction (HMI) and User Experience (UX) – The concepts of HMI and UX are closely related, although they’re often used in different contexts. HMI is a term commonly

¹³ The concept of SOLID, as well as many other programming best practices are found in Robert C. Martin’s “Clean Code”

¹⁴ Introduced in “Managing the Development of Large Software Systems”

¹⁵ Introduced in the Agile Manifesto, signed by several well known software architects

¹⁶ More information on Scrum in “Agile Software Development with Scrum”

used in factory settings to refer to analog controls, screens, levers, and any interactable element with which operators, engineers, and managers may interact in order to visualize or control parameters in machines. With a greater focus on digital consumers, UX focuses on efficiency and ease of use of digital platforms. In other words, HMI focuses on enabling connections between machines and humans, while UX is centered around making those interactions as efficient and easy to use as possible, considering the human's needs. As mentioned in Chapter 1.3, systems that are cumbersome to use, hard to access, and difficult to understand are often abandoned and represent a sunk cost for the company.

2.3 Organizational Context

Across the organization, teams at the CWC strive to work efficiently and provide high-quality products to the main factory. As mentioned in section 2.1, doing so requires handling material through a complicated traffic management process. In this section, we will discuss the primary stakeholders, for this study, as well as the relevant system characteristics needed to be modeled. The two primary stakeholders involved in this project are the AGV management team and the industrial engineering team. Factory operators and company management are classified as secondary stakeholders.

Stakeholders

Achieving full automation is not feasible without an efficient scheduling system. The ideal output from this project involves not only developing a scheduling system from the ground up but also a direct link between said scheduling system and the vehicle management system (VMS) to enable direct publication of moves to the system. The AGV team possesses expertise in automation systems along with the tools and systems necessary to integrate with the broader company environment.

Conversely, the industrial engineering team possesses deep expertise in material flow and process optimization. Their participation in this project is crucial as they possess knowledge of

data-related inquiries. To leverage collaboration, the solution provided in this study should provide factory visibility and much-needed predictive tools for industrial engineering processes.

Although not directly involved in the project's development, plant managers and operators are impacted by the outcome of this study. Plant managers are primarily involved in high-level decision-making, budget allocation, and overall factory stability and improvement, which enabled the conception of this study. Implementing a new system that can lead to significant cost reductions and increased efficiency is naturally relevant to plant managers; thus, the bottom-line impact of implementing a DES system must be clearly communicated.

Factory Operators provide low-level information on the intricacies of the system and could encourage or deter the adoption of a scheduling system. Current material flow operations are agile and efficient as operators move material between cells without restrictions, yet this has resulted in frequent traffic congestions and production inefficiencies. To encourage adoption, the system must enhance, not hinder, operator efficiency by providing clear, real-time scheduling that minimizes disruptions.

Layout

The factory builds airplane wings with composite material, hence the factory name (Composite Wing Center, or CWC). The general process of composite manufacturing requires curing the material to have it keep its final shape. Curing parts involves introducing them into a high temperature oven, designated the name of "Crucible" for this study. Three main part types are produced in the building: "Alpha," "Beta," and "Gamma"¹⁷.

¹⁷ Names of parts and stations have been changed to conform with non-disclosure agreements with the company

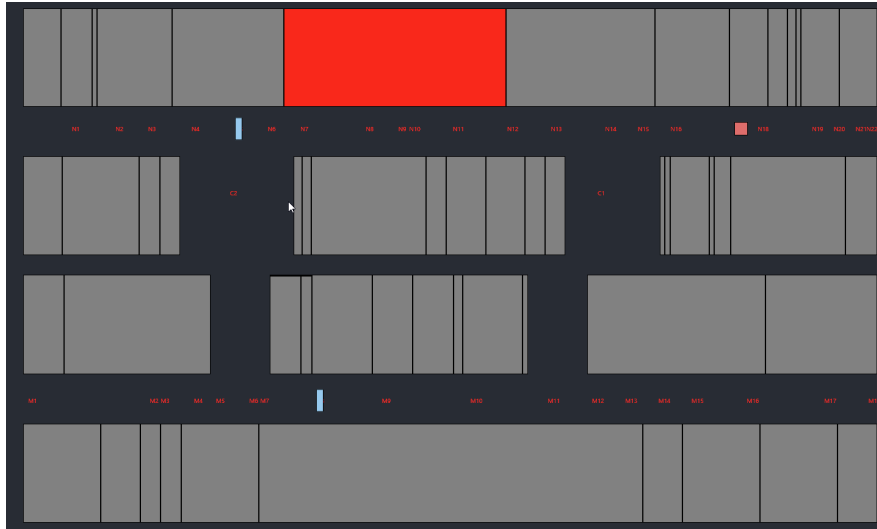


Figure 4 - Factory layout as seen from a birds-eye view. The “Crucible” station, which receives parts of Alpha, Beta, and Gamma, is highlighted in red. Each gray block represents a station. The stations on the top half of this picture perform post-cure operations, whereas the stations at the bottom half of the picture represent pre-cure operations. The red markers on this image represent “nodes” in hallways through which transporters move to transfer parts from one station to the next. As the Crucible is required for the production of all part types, handling traffic in the nodes adjacent to it has presented a big challenge. The horizontal hallway in the middle of the image is not used for transportation.

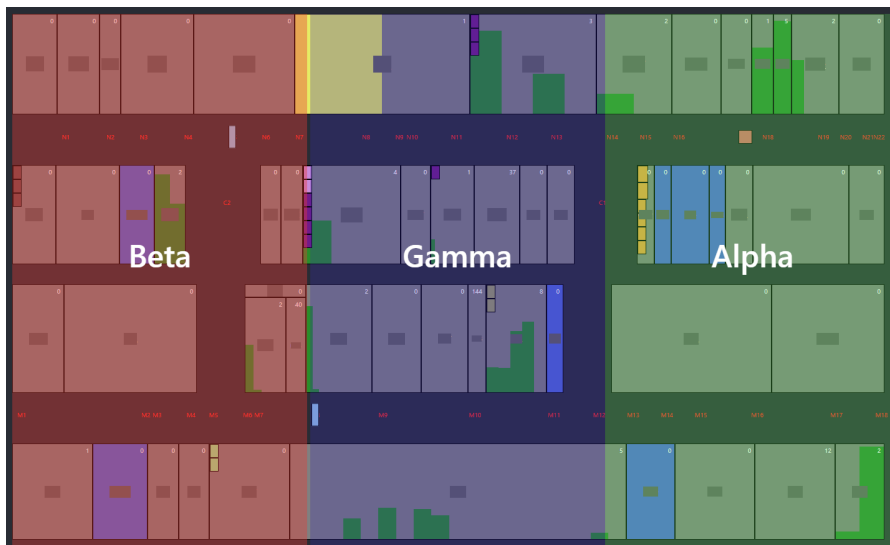


Figure 5 - Alpha, Beta, and Gamma part process locations. This picture complements Figure 4 by depicting the general locations for the work centers that handle Alpha, Beta, and Gamma parts. The section highlighted in green represents stations primarily focused on the production of Alpha parts. As such, traffic in these areas will only affect the production of Alpha parts. The same is true for the stations highlighted in red and Beta parts. The factory section highlighted in blue will perceive the most traffic due to two main reasons: The Crucible station is located here, and Gamma parts represent the largest volume of production.

2.3 Key Terminologies, Definitions, and Assumptions

Producing an effective scheduling solution requires understanding the intricacies of the production system. After conducting several interviews with key stakeholders, we compiled the following list of terms that are necessary to understand and model the production process and design a solution for scheduling.

There are five main categories worth discussing in this section: Parts, stations, equipment, policies, and constraints. We will define the main part types, as well as the particular production requirements necessary for producing them. Then, we will discuss some stations that handle particular processes worth discussing. It's important to note that although there are over 40 stations in the system, we will only discuss those that significantly contribute to traffic management. Next, we discuss the equipment necessary to move parts from one station to another. We discuss production and transportation policies currently implemented in the system for efficient production. Finally, we talk about general system constraints that introduce system limitations.

Parts

Alpha Part – One of the three main components produced at the factory, available in four distinct variations. For this study, we will refer to them as “Alpha-AX,” “Alpha-AY,” “Alpha-BX,” and “Alpha-BY.” Before curing, these parts consume a specific set of Beta components. After curing, Alpha parts, which are detached from an Atlas tool at the Liberis Station, require a Monorail system for moving. Understanding how these different variations behave is essential for accurate simulation.

Beta Part – One of the three main parts manufactured at the factory. These parts are also produced in four variations: “Beta-AX”, “Beta-AY”, “Beta-BX”, and “Beta-BY”. Beta parts go through some stations multiple times during a single production run. This implies that the flow of Beta parts resembles the following pattern: A->B->C->B->C->... where A, B, and C represent stations. This is particularly challenging for traffic planning. Accurately simulating this process

necessitates tracking which point in the process a part occupies, an insight that cannot be easily gained by knowing the station in which the part is located.

Gamma Part – One of the three main parts manufactured at the factory. With over 70 variations, the parts are divided into three main categories: Gamma-A, Gamma-B, and Gamma-C. Each of these categories has a slightly different flow through the factory. Gamma-A is the most common type of part and has a specific pre-cure flow. Gamma-B parts follow a different pre-cure path, but once they reach the cure stage, they follow the exact same flow as Gamma-A parts. Gamma-C parts are quite rare and have been intentionally excluded from this study. Gamma parts consume fragments that are used for their initial processing. Understanding all 70 variations, fragment consumption, and the pathways followed by Gamma parts in detail is essential to developing an accurate simulation.

Stations

Crucible Station – All three types of parts go through the same Crucible Station. The factory has two available Crucible Stations. Prioritizing Crucible utilization is essential. Scheduling of other processes may vary once parts arrive at the Crucible on their scheduled time. Additionally, the Crucible is situated in a high-traffic area (see Figure 4), so movements in and out of the Crucible must be prioritized. As mentioned earlier, performing batch operations at the Crucible is also mandatory. When moving four sets of Gamma parts into the Crucible, four AGV moves are scheduled. Different policies govern these moves. Utilizing four AGVs for the transports is not feasible due to limited availability. Furthermore, to maximize utilization, there is a high likelihood that moves out of the Crucible will occur just before new entries. In this scenario, it is important to consider using the same type of AGV for both entry and exit. Choosing the right policy and implementing it through rule-based algorithms is necessary to compute an effective planned move set.

Nintu Station – Gamma parts attach to an Alpha part at this station. This station has an inventory of Gamma “cassettes.” There are 12 different cassettes, with 2 sets of each, totaling 24 cassettes in the inventory management system. A cassette should be replenished as soon as

possible. The Gamma inventory and Nintu cassette inventory are regularly evaluated. If a cassette can be constructed from the available Gamma parts in inventory, and the Nintu station has an available slot for the cassette, it is then constructed and delivered to the Nintu station for future use. Moving cassettes into the Nintu station requires the use of AGVs, which affects AGV resource usage and contributes to traffic. Therefore, understanding and simulating these rules is crucial. Moreover, the Nintu station may be blocked due to a lack of cassette availability.

Equipment

Transportation Equipment - There are two types of AGVs. AGV-As move Alpha and Gamma parts, while AGV-Bs are exclusively for moving Beta parts. After curing, Alpha parts are transported using a monorail, Gamma parts are moved with Gamma-carts, and Beta parts are conveyed using Beta-carts; therefore, AGVs are no longer utilized. AGVs do not access the monorail section; conversely, the monorail cannot reach most areas the AGVs access. These physical limitations are crucial when implementing path-planning algorithms for scheduling systems.

Atlas Tools – Pre-cure parts are mounted on Atlas Tools. Each type of Alpha part has two sets of Atlas tools. This means that at any given time, there can be up to two of each kind of Alpha parts in pre-cure processes. Beta parts follow the same constraint, including two sets for each of the four configurations. Gamma parts, on the other hand, include 19 Atlas tools, each with a different configuration. These parts occupy physical space and are a limited resource in the factory, so scheduling moves requires understanding the available tools and their locations. Often, these tools are moved into a storage area to make way for other parts to enter the cell. It's crucial to know where each of these tools is located before initiating the construction of a new part.

Additional Tools—Apart from Atlas tools, Hermes Tools are used to move parts using a crane during the post-cure process. Using Hermes tools requires moving the crane to the Hermes storage station, performing the required move, and then dropping off the Hermes tool at its

storage location. A single Hermes-required move involves three moves, each of which will affect scheduling and timing.

Policies

Path Selection - AGV automation depends on magnets installed on the floor. Due to the size of the parts moving through the factory, usually, only one part can move in one lane at any given time. This means the lanes are one-way, requiring parts to wait for others to pass before they can proceed. If possible, parts should take an alternate route. Some movements on specific transporters have constraints, meaning they can only navigate through designated areas.

Push-Pull Stations- Depending on the desired production rate and the maximum speed of each station, stations are classified as either “push” or “pull.” Stations that require nearly maximum utility are known as “push” stations. Push stations must create space for incoming parts immediately after one part is processed. In other words, parts should move out of push stations right away- or as soon as possible- to allow other parts to be produced in that station. In contrast, pull stations typically produce at a rate exceeding the required production rate. This allows these stations greater flexibility in their utilization. Since pull stations don’t need to produce continuously, they aren’t necessarily required to move parts out. Push stations may compel parts out of pull stations to prevent pauses in production. Defining which stations are pull and which are push is crucial for the development of this study.

Buffer Station Planning - Intermediate stations, known as buffer stations, are placed throughout the factory to ensure that push stations can continue production. These stations serve to hold idle parts while waiting for capacity to become available at their next station. Pull stations can act as buffer stations if the subsequent station in the sequence is unavailable. If a station is full and contains an idle part while a new part attempts to enter, the idle part should be moved to an external buffer station to make room for the incoming part. It is crucial to incorporate buffer stations in the development of the scheduling algorithm, as they entail additional moves within the factory that may not have been initially planned.

Gamma-A First Station - The first station in the Gamma-A process introduces particular complexity to the task at hand. For starters, the entire layup process was restructured in a new location within the factory, following completely new logic. This means that, by the time I began working there, there was no data related to this new station. The new station comprises 19 lanes, each linked to a specific set of Gamma-A parts. Each lane can process a particular set of 2, 4, or 6 Gamma parts mounted on an Atlas tool; examples of such configurations can be seen in Figure 6. Consequently, each Atlas tool is unique to its associated set of Gamma parts and lanes. After curing an Atlas tool with a set of Gamma parts, it is free to return to the Gamma-A First station and start processing a new set of parts. These capacity constraints and part configurations are essential in developing an accurate digital twin of the factory.

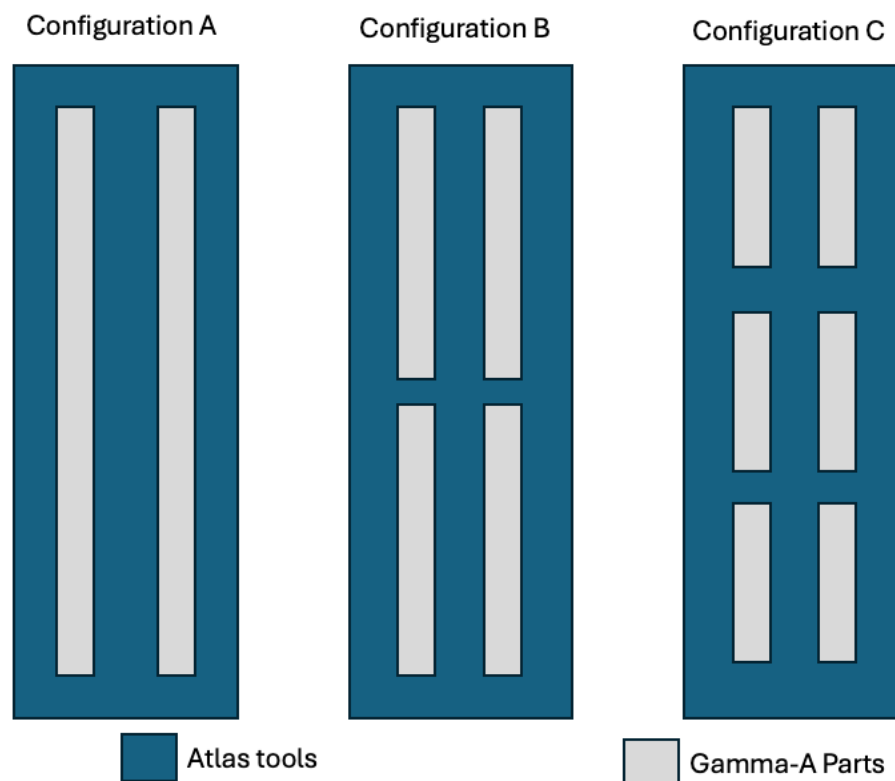


Figure 6 - Different Configurations for Gamma parts during the pre-cure process. This diagram is a visual representation of how parts Gamma parts can be configured on Gamma-A First Station. On atlas tools of essentially the same size, either 2, 4, or 6 Gamma parts can be placed.

Gamma Split-Join Process – The first part of the Gamma process occurs on Atlas tools, which require AGVs to be moved around. Once the LM tools leave the Crucible Station, the Gamma

configuration changes; half of the Gamma parts initially in the Atlas tool are mounted in one cart, while the other half are mounted in a different cart. The Gamma parts undergo additional processing until they reach storage, where they are kept as individual Gamma parts. Gamma parts are then combined in various cassettes, which have specific Gamma configurations. Parts mounted in an Atlas tool are processed together. Similarly, once transferred to the carts, a single cart is processed as one, and finally, cassettes are processed as a single fragment to be consumed by a Nintu station.

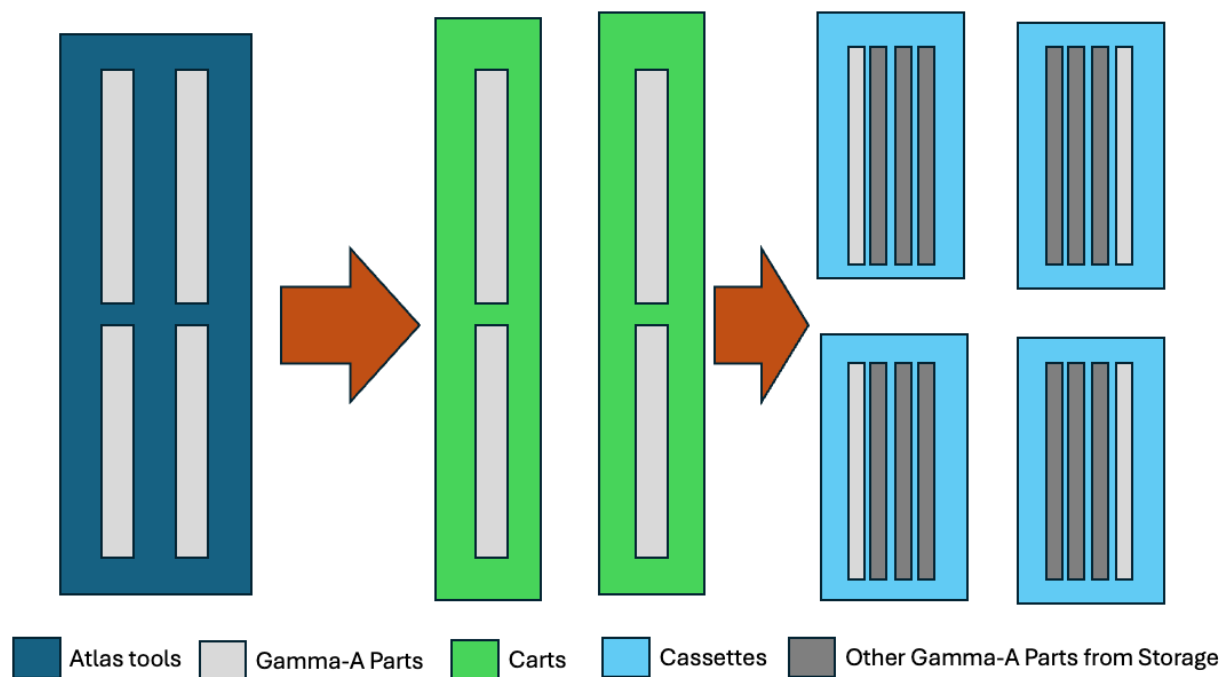


Figure 7 - Gamma splitting process. Pre-cure parts are mounted in an Atlas tool. They're split into carts and, finally, placed into inventory. Finally, when ready to move, they're placed among other Gamma parts on cassettes.

AGV idle policy - Once an AGV set completes a move, one of three things can happen: A new team of operators takes over the AGV to perform another move in a nearby station, the AGVs are kept in the hallway to be used soon by another station, or the AGVs get driven “home” to get charged. Deciding what to do when an AGV goes idle is extremely difficult to do, because you have to predict if it’s going to be needed in the near future nearby, but you also have to evaluate if the move it’s going to perform is viable with the current AGV battery level. One possible policy is just taking the AGV back immediately when a move gets completed, unless

someone requests the AGVs mid-move, creating a queue of future moves. This is what operators currently do. Another possible policy is waiting at the destination for a fixed amount of time, to see if any moves come up, and then drive home. The optimal strategy requires the system to know exactly what station is going to require that specific set of AGVs next, and move them straight there to be used.

Constraints

Shifts—Operators work three shifts and avoid making moves near the end of their shift. Certain parts of the process require three shifts, while others require only one or two shifts. In general, operator availability is a systematic constraint that needs to be considered in this study.

Batch Operations – Certain operations must wait for others to finish before proceeding. A simple example is the processing of Gamma parts in the Crucible station. Before moving the Atlas tool into the Crucible, it's crucial that all Gamma parts remain in a buffer station until a complete batch of four Atlas tools is prepared to move into the Crucible station. Similar to fragment consumption, batches are a necessary implementation when scheduling moves, as they may prevent moves until a completed batch is ready.

Capacity Limitations - Each cell has a specific capacity. It can hold only a limited number of parts at any given time. Before deciding to transfer a part to its next cell, the cell's capacity must be assessed to ensure it can accommodate the incoming part.

Fragment Consumption - Some stations require multiple parts before they can begin processing. The output of these stations is a single part. This occurs in several parts of the process, with some stations needing two or three fragments to be combined into a resulting part, while the Nintu station combines an average of 12 Gamma parts, represented as fragments, into one Alpha part. In this context, a fragment is a physical element that will be consumed by another part. Since fragments are essential for processing, their consumption logic is crucial to simulate stations as blocked when the necessary fragments are unavailable.

AGV battery life – AGVs' battery life gets depleted as they perform moves. Battery depletion happens mostly when AGVs move or perform a move. When AGVs move to their "home" station, they get charged. It's important that AGVs' batteries don't get depleted while they're moving. Therefore, AGVs should be moved into their charging station as soon as they cross a certain battery threshold. Once this happens, AGVs are considered unavailable until they're fully charged.

Chapter 3: Methods

3.1 Overview of the Research Design

This study follows a simulation-based research approach. It aims to optimize the scheduling of moving parts such as AGVs, cranes, and carts and improve traffic flow by simulating material movement. The first task for this project involved conducting a series of interviews to understand the material flow through the factory, the policies used for selecting and moving resources and defining a potential solution. Data was collected to create an accurate real-time representation of the factory. DES software was selected after evaluating the available options, and finally, the simulation was assembled, validated, and deployed.

Weekly meetings with the AGV team and the industrial engineering team produced insights into this project's requirements and possible implementations. These interviews also formulated an understanding of the data structure and future implementations that could produce efficiencies. They also allowed us to understand the limitations of the data as well as the limitations of the produced simulation with relation to real factory behavior. Business rules for decision-making were extracted from subject matter experts (SMEs) at the company. They were compiled into a single document and later implemented as rule-based algorithms in a structured coding system.

Deciding what DES system to use consisted of evaluating the possible options and weighing the advantages and disadvantages of each. Next, small prototypes were developed on the most suitable contenders: SIMIO and SimPy. Finally, SimPy was chosen over SIMIO because it allowed for custom logic integration via Python, seamless API connectivity, and better adaptability to the company's existing systems although it requires extensive knowledge of Python programming and lacks a user interface. To properly accomplish the goals of this research using SimPy, a simple front-end software was developed to generate the visualization of the simulation, as well as control certain parameters. This front-end was developed using React.JS, as it's one of the most widely adopted web frameworks. The GUI allows users to visualize real-time AGV movements, adjust scheduling constraints, and modify resource allocation rules.

Connecting the SimPy simulation with a front-end requires the development of a simple back-end API server. Flask was selected as the Python framework for this task. Connecting Flask and SimPy together is a trivial process, a clear example of SimPy's advantage for interconnectivity and expandability over SIMIO and other more rigid software.

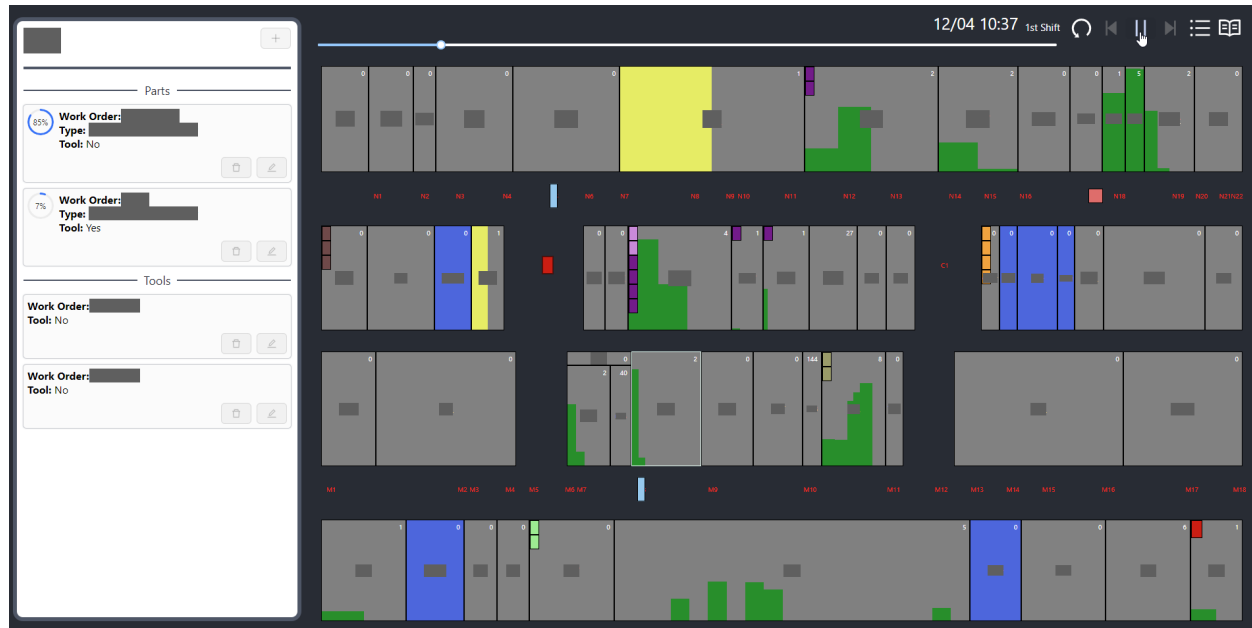


Figure 8 - Screenshot of simulation system front-end application. This screenshot shows a birds-eye view of the factory floor's current state. The green bars on each station indicate the progress level of a part in that station. Yellow stations indicate parts that have been completed and are waiting to be moved. The left panel shows additional details of the contents of a given station. The top section controls allow users to run and visualize the simulation for a total of 10 hours. Other components such as cranes, AGVs, carts, and tools can be easily inspected in the simulation. Station names have been redacted with a gray block.

Finally, once the software suite has been chosen, the simulation is built, making sure all the business rules are in place and an easy-to-use GUI is developed. This is an iterative process that requires developing rules step by step and validating them with stakeholders. This step also consists of selecting a method for deployment. In our case, this method involves hosting a web application on the company's internal servers. Once all business rules have been implemented in code, a validation step consisting of both historical and real-time data took place. Historical validation was used to measure the difference between predicted future states – those generated by the simulation - and real future states – those coming from the data gathered – and correct parameters in the simulation. Real-time validation allowed cross-checking the output of the simulation with real physical parts in the factory and uncovered issues with assumptions made during the development of the system. Validation accuracy was assessed by

comparing simulation-predicted AGV movements with actual recorded movements, using metrics such as scheduling accuracy, AGV idle time reduction, and congestion improvements.

3.2 Data Collection

Creating an effective digital twin relies on capturing the factory's state at any moment.¹⁸ Thus, identifying reliable sources of information within the company is crucial. This study identifies four key data sources in the factory: An automatic data gathering system on certain machines, a factory-wide RFID system, an AGV vision system, and an operator stamping system. We explore the feasibility of each data source, as well as how the data must be extracted, cleaned, filtered, and combined to accurately reflect the factory floor.

The automatic data gathering system represents a significant advancement toward Industry 4.0, as it publishes automated data about machine performance and usage. Although this system is very powerful, it still requires human input. More importantly, this automated system is available on only a subset of machines – approximately 20% of machines in the factory – making it unreliable as a source of truth for obtaining a comprehensive understanding of the factory's state.

The factory's RFID system was configured to track the positions of AGVs at any given time. Theoretically, this RFID system can be used alongside any type of RFID tag to detect the positions of elements within the factory. This system has great potential and should be further explored in the future. These RFID systems are also challenged by potential damage caused by the high temperatures experienced at the Crucible station. These systems could provide accurate location information, yet progress data would still need to be gathered elsewhere.

The AGV vision system was briefly explored as a means of detecting which tool was mounted on the AGV. This would yield partial information about the part being transported when an AGV makes a move. However, this system was never fully configured and would not have served as

¹⁸ Refer to chapter 2.2 for detailed definition of digital twins, and to Cyngn's article "Applications of Digital Twins in Manufacturing" for more information.

an ideal source of information even if it had been. By process of elimination, stamps emerge as the only viable source of data, though they are not without their flaws.

Serving as the primary source of truth for determining both the location and progress of a part in the factory flow, stamps are the key data source for this research. Operators manually input the stamps into a computerized system. There is a fixed set of stamps along with a planned list that must be completed to finalize a part. Stamps are subject to potential human error as operators must register them in the system periodically. Alpha parts follow two lists of stamps, also known as “orders.” During the pre-cure process, an order is followed until reaching the Nintu station, which marks a “completed” order; thereafter, a new order is initialized. Delivery to final storage concludes the second order for alpha parts. Beta parts adhere to a similar structure, with two available orders- one for pre-cure stamps and another for post-cure stamps. Lastly, Gamma parts follow six orders: two orders for Gamma-A fragments, two for Gamma-B fragments, one for Gamma-A parts, and one for Gamma-B parts. Understanding the logic behind how this information is captured is essential for constructing adequate data analysis and filtering necessary to replicate the current state of the factory.

Another drawback of using stamps as a source of truth for part movement is the lack of stamps configured for part movement and the absence of a defined stamping process for managing buffer stations and Atlas tool locations. In other words, stamps are only approximately reliable for confirming the location of a part or tool, and as such, their locations must be inferred using a set of rules. Furthermore, the distribution of stamps is not linear. In other words, one stamp may represent 50% of the time required to complete a task, while others might affect less than 1% of the total process time. All of these issues must be considered when building a digital model of the factory, as ignoring them would lead to an inaccurate model.

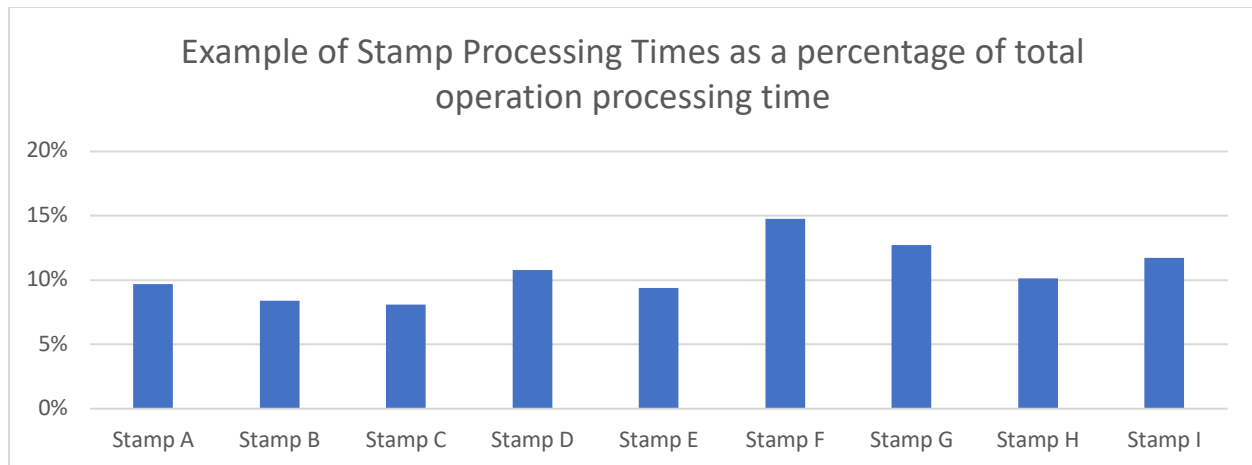


Figure 9 - Example of stamp processing times as a percentage of total operation processing time. This example serves as a visual explanation of stamp processing times, where some stamps (like stamp F) may take up a significant amount of time to process, in contrast to other stamps, such as stamp C.

Aside from the intrinsic issues with using stamp data to approximate factory state, external factors must also be considered to reduce noise and irrelevant data. Changes in factory layout render prior data unusable, as processing times, stamps, and transportation logistics may vary. A recent example of this is the introduction of the Gamma-A first station¹⁹. At the beginning of this project, the production team was conducting tests to implement new machinery and logistics that were not only situated in a separate physical location but also introduced a completely new process for which limited data had been collected up to that point. Consequently, there was very little usable data for the Gamma-A first station. Additionally, performing data filtering and cleaning is a common requirement when querying internal data. Integrating these considerations involves removing unwanted records, filtering certain dates, using advanced grouping queries, and, importantly, having a deep understanding of the relational database structure.

Interviews conducted with the industrial engineering team yielded key qualitative and quantitative information on the process flow. These interviews also revealed the need for data expertise in manufacturing environments. The qualitative data from these interviews, which included the intricacies of the factory flow, the engineering pain points, and the main operational struggles, guided the efforts in developing a viable solution. Communicating those

¹⁹ Refer back to section 2.2 for more details on this station's function.

needs guided the development of features in the front-end and back-end of the final product. The interviews also allowed for the standardization of processes that originally functioned through an “it-depends” methodology.

It is important to note that secondary data sources, primarily found in Excel documents shared among various stakeholders at the company, contained valuable information for developing the simulation in this paper. One such document includes priority lists that indicate which parts should be manufactured first. Other examples include data related to the combination of Gamma-A parts on different Atlas tools, the integration of those parts into various cassettes, time studies conducted at the company, and the required cassettes for consumption in the Nintu station for a specific variation of the Alpha part. These documents, essential for accurately developing a simulation of the factory, were created through discussions with different stakeholders.

With the raw data from stamps and the considerations mentioned above, the data was cleaned using a combination of Structured Query Language (SQL) queries and the Python library “pandas.” The query developed for this study utilized various grouping and joining instructions to clean the data. It employs a reference date and aims to extract the factory's state on that date by disregarding stamps that were applied after it. For real-time simulations, the date would be the current one; however, for validation and testing, a past date is necessary. The output from the query is detailed to the operation level, categorizing stamps into two groups: Completed stamps and total stamps. Dividing completed stamps by total stamps yields the percentage progress in a given operation. While operations ideally correlate with a station, this is not always the case. To further combine stamps and disregard unwanted operations, a Python filtering algorithm was developed. To ensure data reliability, we applied a bottom-up approach for determining the position and percentage of a part. For example, if a part following the flow A-B-C-D was identified with respective percentage progresses of 100%-90%-20%-0%, our system would identify that part as being in station C, the last station with some stamp present. Conversely, if the percentages were 100%-90%-20%-1%, the system would place the part in station D. Furthermore, records were validated using a set of predefined rules, such as

ensuring part sequences followed expected workflows. The output of this Python algorithm is a data structure that can be seamlessly integrated into the simulation.

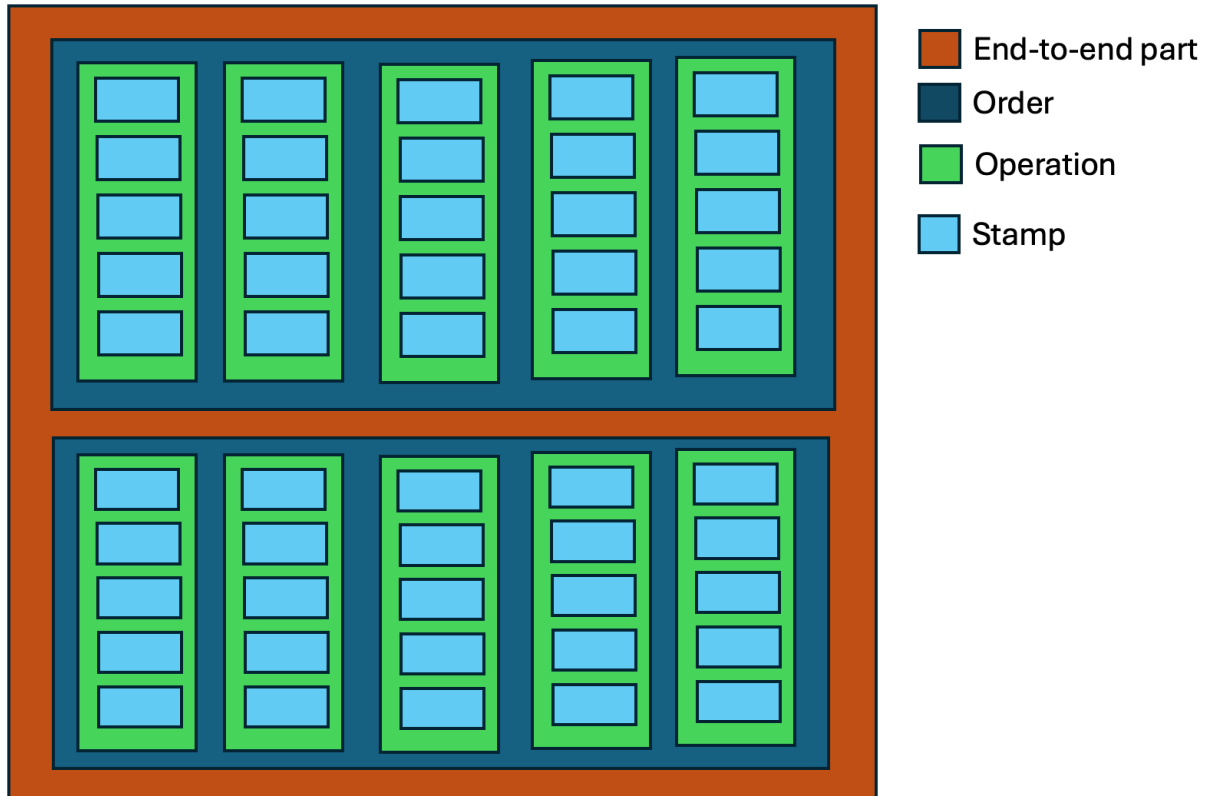


Figure 10 - Visual representation of the different logical levels of data. As an example, one Alpha part, represented in orange, has several “orders” associated with it. Each one of these orders has several operations, and each operation has several steps (or stamps). This diagram is a visual representation of how the data is structured and used to calculate a part’s progress.

3.3 Tools, Frameworks, and Models

The development of our solution involved evaluating and selecting the appropriate tool for Discrete Event Simulation (DES), constructing an accurate model, and deploying it to its users. To achieve these goals, we first conducted a comparative analysis of the DES software services. Next, we assembled a comprehensive stack of tools necessary for a viable solution. Then, we employed Agile iterative processes to develop a robust solution through small increments. Finally, we determined the best approach for deploying the tool.

Finding and selecting the right software involves exploring the available options, their features and capabilities, and how they align with the specific environment and use case. First, we categorize the software into two groups: licensed software suites and open-source libraries. Given the active license with SIMIO at the company, SIMIO was the clear choice when considering licensed software. It offers an interactive GUI, making it accessible for non-programmers, yet it is powerful enough to solve any DES problem. SIMIO also provides several API-ready integrations, built-in optimization, and metric-gathering solutions.

Software	Flexibility	Learning Curve	Integration Capabilities	Licensing
<i>SIMIO</i>	High	Moderate	High	Active License
<i>SimPy</i>	Very High	Steep without programming background	Very High	Open Source
<i>AnyLogic</i>	High	Moderate	High	Required
<i>FlexSim</i>	High	Moderate	High	Required
<i>Arena</i>	Moderate	Moderate	Moderate	Required

Figure 11 - Comprehensive comparison of simulation software. This table represents the comparison used to determine which simulation software was best suited for our application.

In contrast, open-source solutions for DES exist in most popular programming languages. An internal company analysis revealed Python as the most widely adopted programming language among the engineers. Within Python libraries, SimPy is the most widely adopted and hosts the largest community, which provides an advantage on debugging and solution design. SimPy also has the advantage of being a pre-approved software at the company. Moreover, two of the most widely used back-end frameworks (Django and Flask) use Python.

Selecting between SIMIO and SimPy consisted of understanding the use cases and constructing a Proof of Concept using each software to understand their capabilities and limitations. Although SIMIO could potentially provide useful insights in traffic management and factory simulation, its implementation for our use case resulted in convoluted and unnecessarily complicated. Simulating over 70 distinct parts in SIMIO led to excessive computational overhead and cluttered visualization, making it difficult to track entities in real time. One of

SIMIO's main advantages is its leverage of powerful stochastic variables and optimization algorithms, both of which are unnecessary for our solution. Moreover, other requirements, such as part combination and database connection, were difficult to implement, and no subject matter expert (SME) familiar with the factory flow was knowledgeable enough in SIMIO to suggest a solution and streamline the development. Similarly, implementing a central control system to schedule moves and decide which part should move where wasn't easily achievable using SIMIO. Although SIMIO is an extremely powerful software, it's best suited for applications where stochasticity is required, and the system doesn't require as many special cases.

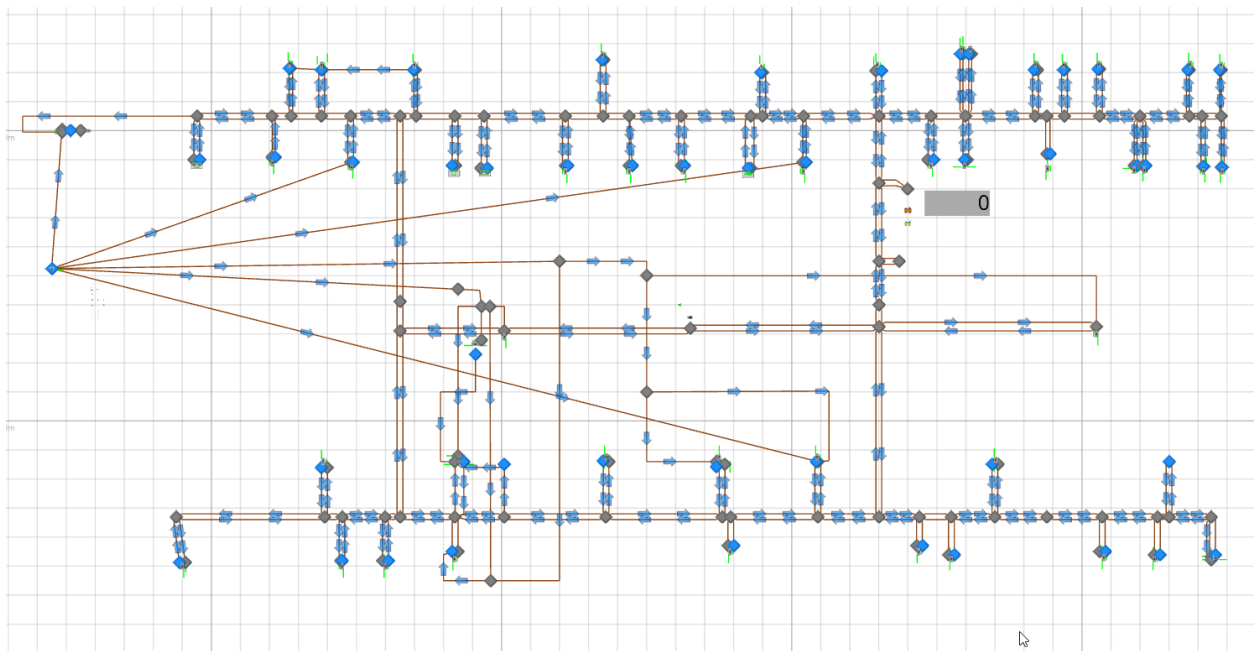


Figure 12 - This screenshot shows the setup of the CWC process in SIMIO's simulation interface. The development was not completed, but this screenshot shows the complexity of the simulation.

As pre-packaged simulation software has proven to be suboptimal, the only remaining option is to develop a SimPy Proof of Concept to ensure that the goals of this study can be achieved. Otherwise, a new methodology must be designed. Utilizing SimPy requires integrating other pieces of software into the stack, such as a back-end framework and a front-end application. The Proof of Concept for SimPy did not include a visual aid. Instead, the library was explored, and a small simulation with command-line output was developed. This Proof of Concept examined, on a smaller scale, some of the challenges that were difficult to model with SIMIO.

The simulation involved modeling AGVs with decreasing battery rates, moving through nodes to enter stations. We successfully implemented a path-planning algorithm and designed a part combination and consumption system, both of which were not easily achievable with SIMIO. SimPy offered superior adaptability for implementing a central control system to dynamically schedule moves, something that was cumbersome in SIMIO due to its predefined event-based structure.

With a successful Proof Of Concept for our DES software (SimPy), the next task consisted of selecting the frameworks and tools for the rest of the stack. The user requirements defined in this study demand the implementation of a flexible, powerful, and easily accessible application with a visual interface. User interfaces can be built natively for desktop and laptop computers, for mobile applications, or as web applications. The web front-end framework React.JS is powerful enough to enable quick, agile development of components for web applications. Additionally, through minimal code changes, React.js applications can be used in desktop or mobile environments. These reasons guided the selection of React.js as our front-end framework. Harnessing a stand-alone front-end application, in turn, requires the design and deployment of a back-end API. Out of the two most popular Python frameworks for API development, the company had pre-approved Flask. Flask's lightweight, modular nature allowed for rapid API development with minimal overhead, making it an ideal choice for integrating with the simulation engine and front-end. Additionally, it avoids the bureaucratic processes of obtaining approval for Django. Once the application was built, testing on different devices was possible using Electron, a wrapping library for React.js that allows the building of desktop applications from the same codebase. This methodology is used in a variety of software products, allowing developers to centralize a code-base, mitigating errors and increasing efficiency.

To enable the final product to be easily expanded, this study requires taking into consideration future tools. First, the web application should be ready to be deployed to the company's internal servers for factory-wide usage. The previous architecture permits this integration seamlessly, as long as full access to the server is granted. Moreover, the Vehicle Management System (VMS) to which the AGVs connect must also be considered. Although currently not

connected to the internal network, future iterations of this project will require sending commands directly to the VMS from the simulated solution.

3.4 Solution Design

The goal of the solution is to create an accurate digital twin of the factory that enables real-time decision-making, optimizes transport scheduling, and minimizes bottlenecks. This is achieved by structuring the system into modular components, each handling a specific aspect of factory operations. Once the factory's intricacies are well understood, the data gathered, and the problem defined, constructing the solution involves developing a series of functions to run and visualize the simulation. By applying Clean Code principles, it's important to separate data entities into well-defined atomic functions and files while utilizing object-oriented programming whenever possible. Modeling the material flow with SimPy can be divided into six main categories: path planning, transporters, flows, stations, hot-cold start, and utilities. It's also prudent to discuss the design of the API connection and the front-end design.

3.4.1 Path planning

It is essential that the simulation has a way of representing the physical space in an accurate fashion. To do so, a scale blueprint of the factory floor was used to capture the layout. Figure 11 shows a representation of the method used. Overlaid on the blueprint, we mapped out a series of equidistant points, called "nodes." These nodes are, therefore, a scaled representation of the factory pathways. Since we don't require keeping track of every node, we extract only the nodes that are directly in front of a station. Transporters will inevitably stop at those nodes and then enter the station, so it's not necessary to consider intermediate nodes. Figure 12 shows, in the final simulation, the resulting nodes that were used. Using a time study, we extrapolate a trend line that represents the average time it takes for a transporter to move between two nodes. If a shortest path algorithm can be computed, we can then calculate the (simulated) time it takes to get from one place to the next.

Having a mapped set of nodes and the distances between them, we can begin constructing **networks**. Transporters can only navigate within the network they have been assigned. A monorail, for instance, which only travels a fixed area of the map, would have a network that

only includes the nodes in its physical location. This is necessary to avoid issues with path planning. When transporters “seize” a node, this node becomes unavailable to other transporters. This creates an additional filter, or “mask,” that removes all unavailable nodes from the network. The last step of the process is running a shortest path algorithm with the nodes available in the network. For calculating the best path between two nodes, we use Dijkstra’s algorithm, a well-known algorithm for shortest paths of weighted graphs. Figure 13 shows how the path planning algorithm may be implemented. The path-planning module also ensures real-time congestion management by dynamically updating node availability. If a path becomes blocked, transporters can query alternative available routes within their assigned network.

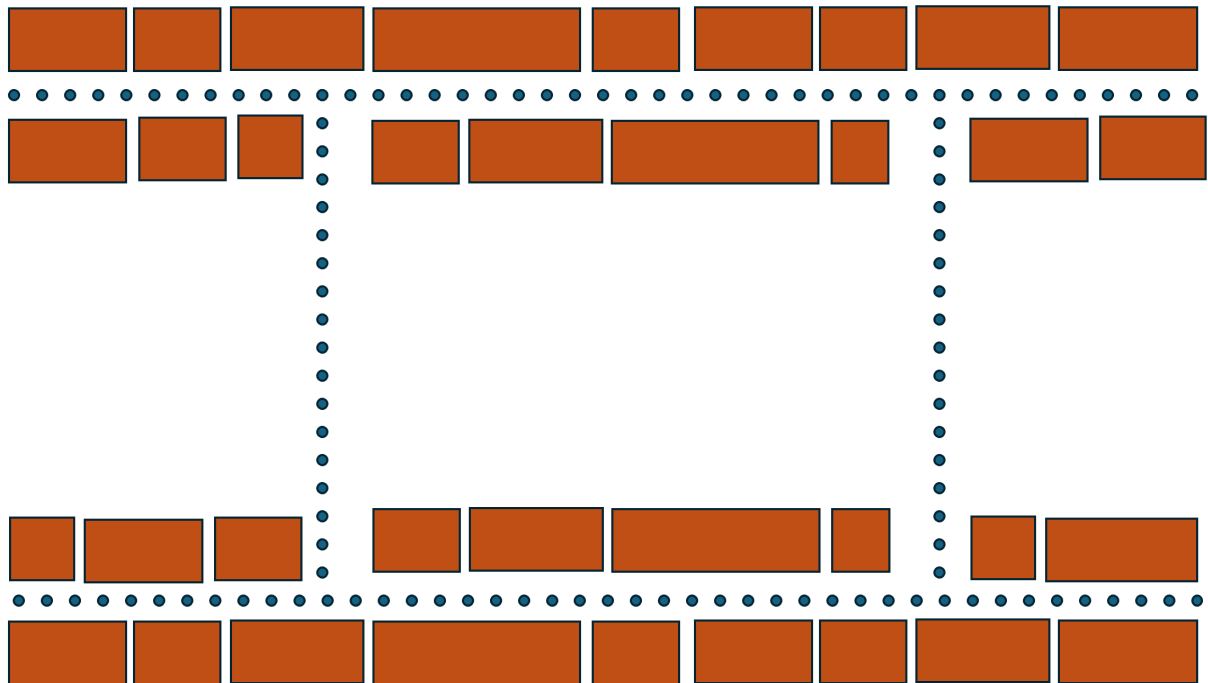


Figure 13 - This diagram demonstrates how nodes are represented on the factory floor. In this birdseye view diagram, stations are represented as orange blocks, and possible paths through nodes are blue dots.

```

function plan_path(src, dst, mask, network){
    nodes = apply_mask(mask, network);
    if (src not in nodes or dst not in nodes){
        throw NodesNotInNetwork;
    }
    path = dijkstra_shortest_path(src, dst, nodes);
    return path;
}

```

Figure 14 - Path planning algorithm. Dijkstra's Algorithm is a traditionally used shortest-path algorithm, which we apply directly here. This algorithm shows how a mask is applied to obtain the available nodes and then the shortest-path is computed.

3.4.2 Transporters

The transporter class was designed to represent AGVs, cranes, carts, and the monorail. The usage of transporters consists of the following functions:

1. **request_path** – Attempts to request a path between the transporter's current position and a destination node. If no path is currently available due to other transporters, this function waits for the path to be available. Once a move is requested, the transporter seizes all nodes in its path to ensure it reaches its destination. This prevents other transporters from entering a "dead-lock" situation. If a node that is in use is requested, the requester is added to a queue which follows FIFO logic.
2. **idle_to_home** – Once a transporter has reached its destination, this function decides what the transporter should do. The options are: Go home, stay on the current node, or wait in the current node for some time, if no requests are received, go home.
3. **move** – Once a path has been computed, the transporter moves, node by node, to its destination. Since the output of the path planning function has already been normalized to simulation time, we use these values to move from node to node in the simulation.

4. **drive_home** – Helper function for calculating the path to the predefined “home” node, and move the transporter.
5. **go_charge** – AGVs require charging once the battery has reached a threshold. Once that threshold is reached, the AGVs will be set as inactive and will move home. They stay there until fully charged.

3.4.3 Flows

Alpha, Beta, and Gamma parts move through the factory in particular ways. They use a distinct set of transporters, follow a certain order of stations, and take a certain amount of time to process at each station. In this paper, we refer to these sets of instructions as the “flow” of each part. The step-by-step process that is involved in fetching tools, waiting for material, waiting for processing, fetching transporters, loading and offloading, and completing a part is defined in these flow files. Every part variation follows its own flow function. And every new part in the system initializes a parallel run of a flow function.

3.4.4 Stations

Physical locations that can hold parts or process them can be considered either Storage or Stations. Stations themselves can serve as storage when processing is complete. This differentiation is important because it consists of two distinct capacities in simulation: Storage capacity and production capacity. **Storage capacity** represents how many items can be physically located within the station. Alternatively, **production capacity** indicates how many parts can be processed in parallel at a station. When a new part starts processing it consumes both of these resources, but once it is completed it no longer uses production capacity, freeing up space for other parts to seize that resource and push the current part out of the station if necessary. Stations execute the work simulation and publish events when the work is completed.

3.4.5 Hot and Cold Start

The entry point of any simulation is creating entities which will interact with the simulation. In our case, those entities are parts and they can be introduced into the simulation in one of two

ways: Using real-time data (hot-start) or using a part generator (cold-start). Initial validation tests were performed without access to the data, and creating mock data resulted in too convoluted work to be worth the effort; therefore, a cold start generator was created. Generators can be configured to create parts at any rate and of any type. Testing and developing using a cold-start method is ideal since it provides visibility on the complete flow of a part. It also enables a controlled flow of parts. Moreover, the cold-start method has several advantages in future usage for station processing time analysis and capacity analysis.

The hot-start method involves the use of either a randomized initial state or a data-fed initial state. For either, the part flows must be configured such that a part skips the steps of the flow prior to the step of the process the part is at. To illustrate this, imagine a part (P1) whose flow is A-B-C-D and has an initial state C(60%), and imagine another part (P2) of the same type and the same flow, whose initial state is B(20%). At time zero, P1 shall skip steps A and B, and jump directly to C, and complete only the last 40% of the simulated process. In contrast, at the same time zero, part B will skip station A and jump directly to station B, completing 80% of the work. Schedule prediction depends on the hot-start initialization utilizing the data gathered previously.

3.4.6 Utilities

Some utility functions implemented in the development of our solution are worth mentioning. First, SimPy's library contains two functions for yielding functions: AnyOf and AllOf. As the name suggests, these functions will wait to continue until either any of the specified processes completes or all of the specified processes complete. Nonetheless, when gathering resources it's necessary to fetch either resource one or resource two, whichever is available first. Say, for instance, a part requires an AGV. The part should send a request to all three AGVs, and select the first one that responds. Using SimPy's "AnyOf" function, all three AGVs are requested as expected, but since only one is used and all three received requests, the other two AGVs remain blocked indefinitely. This motivated the implementation of a "GetFirstOf" utility, which mitigates this issue by releasing and canceling the requests that were not resolved first.

The second utility function worth mentioning is the **push_out** function. Recalling the definition of push-pull stations in Chapter 2, the push_out function executes the necessary steps to

ensure that priority parts are processed immediately, and that idle parts are moved to one of their buffer locations if available. The algorithm used for this function is best explained by Figure 15.

```
function push_out(station, storage_areas){
    capacity_available = station.capacity;
    if(capacity_available > 0) return;
    if(storage_areas.len == 0) return;

    # Getting here means there's something we need to move
    moving_to = NULL;
    for (storage_area in storage_areas){
        if (storage_area is empty){
            moving_to = storage_area;
            break;
        }
    }
    if(moving_to == NULL) return;
    wait for station.idle_parts[0].move(moving_to);
    return;
}
```

Figure 15 - Push-out Algorithm. This algorithm is used to evaluate whether a part can move into the next station or not. If the station has capacity, the part can move in directly. If the station is full, we check if there's an idle part at the station, and we check if the idle part can be moved to a buffer or storage area. If that's the case, we move the part to that station. This algorithm is important for push stations.

3.4.7 API Connection

Information transfer between the simulation and the front-end happens using a REST API. The API enables only two POST endpoints. The first is an endpoint for retrieving the initial state of the system. This endpoint should access the company's internal systems, clean the data as described in Chapter 3.3, configure it as a JSON object, and finally send it to its client. The

second endpoint retrieves the simulation output given a single parameter: The initial state from which the simulation should start. The data used for this endpoint should be chosen carefully, as a heavy payload could cause long waits when retrieving the data. For our example, using a snapshot of the complete state of the factory for every timestep could result in payloads of 50 to 500MB. The chosen design for these payloads consists of a list of curated event strings, which contain only the essential information required to visualize the simulation. These events must be generated across the simulation, paying careful attention to including every single one, as a missing event could cause an error in the front-end that's not replicable in the back-end. By filtering events and removing redundant state information, the payload size is reduced to under 3MB per request, ensuring faster API responses.

3.4.8 Front-end Design

Recalling the main goals of this study, the solution must provide a visual representation of the factory state, must be easily usable, solve traffic management, and predict future moves in order to allow scheduling. It is evident that the development of a simple user interface is required, therefore designing the front-end was done using a standard component library which not only streamlined development but also enabled simple and aesthetic components that users have been proven to have easy interactions with. Several iterations of front-end designs were developed and tested with a team of potential users. To eradicate additional unplanned variability, a component to add, remove, and edit parts in progress was designed. Luckily, this component seamlessly integrates with existing endpoints, since the same simulation endpoint can be called with a slightly different initial state.

Furthermore, two front-end solutions were designed and developed to predict future moves. The first is the simulation player, which consists of a forward, backward, and a play button, as well as a scrolling component to navigate the simulation. These allow for granular analysis of the steps in the simulation, as well as a visualization. The second solution is the display of a "Move Log" that lists all the scheduled moves that will likely happen in ascending order, and the exact time at which they will happen.

Visually interpreting the complete state of a factory in a single screen is extremely challenging when there are over 70 active parts in the system. This was solved by color coding states of stations, with progress bar indicators that inform how far along parts are in that station, without explicitly stating the part. Additionally, implementing a side bar activated upon clicking on a station provided further information on the parts and progress, as well as allowed changes in the station. Although some colors were chosen with standard color theory methods to ensure users' understanding of their meaning, an "Instructions" panel was created for consultations and further queries on the functions available in the software.

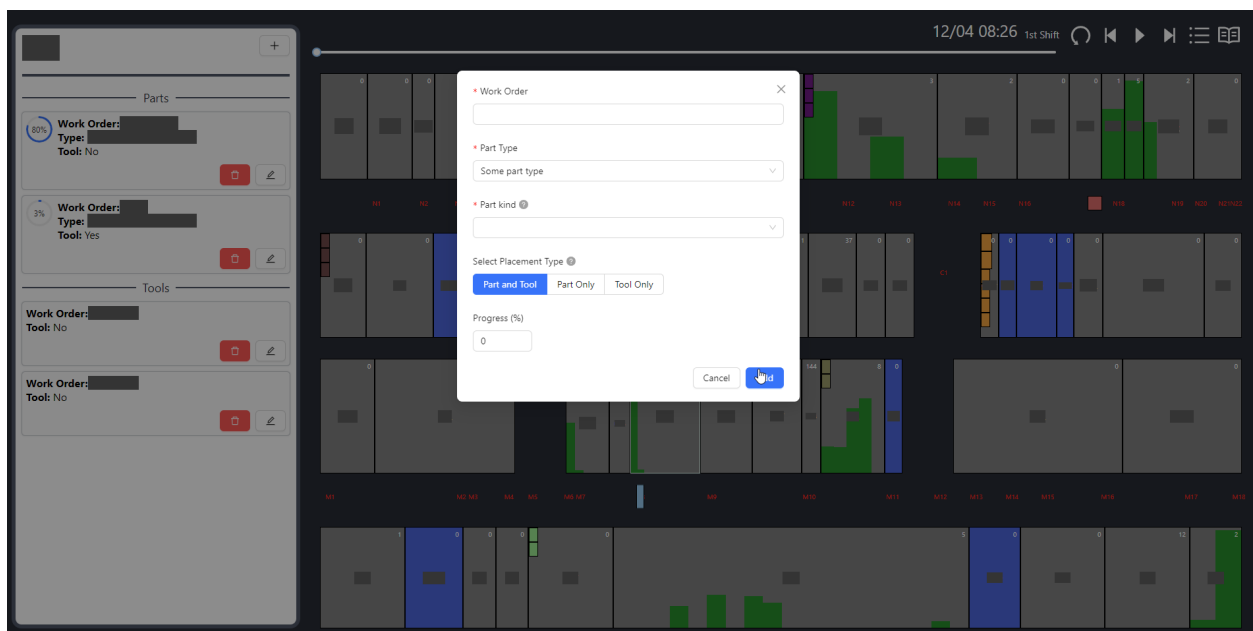


Figure 16 - Screenshot of UX. This panel shows the UX for moving or editing a part. In an ideal case, parts' locations are automatically extracted, yet this is not always the case. This UX was designed to move incorrectly placed parts, or add parts that are not shown in the system.

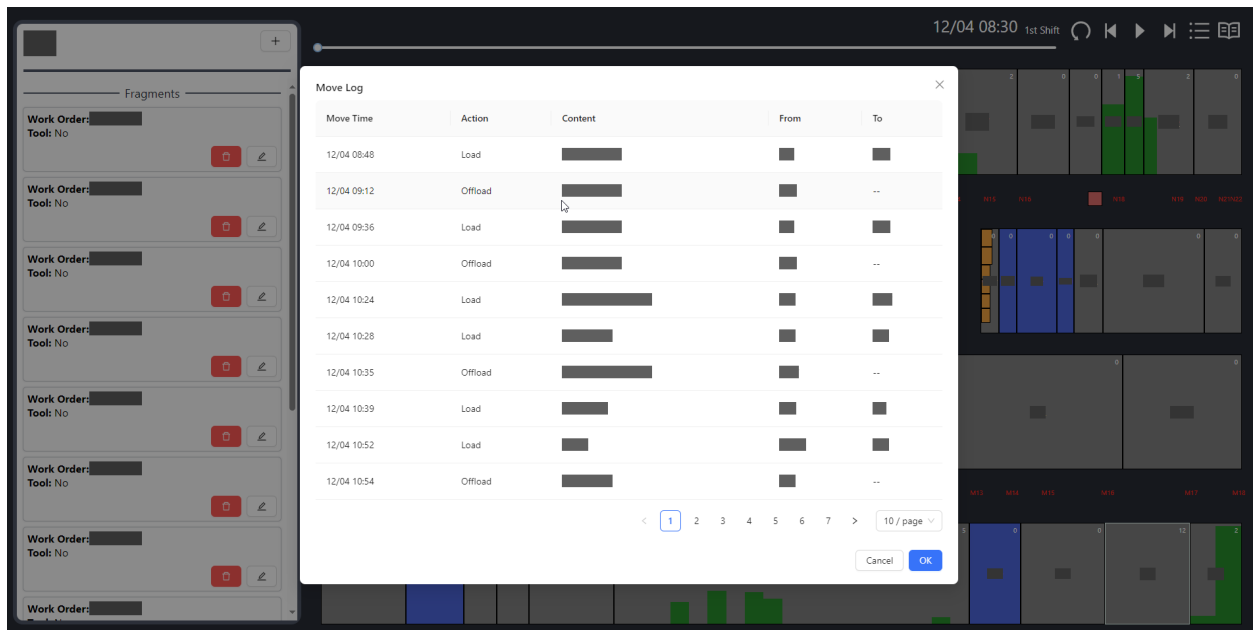


Figure 17 - Screenshot of move log. Simulations in our proposed solution run for 10 hours and generate a list of moves with times and locations. The list of moves is displayed in this panel for easy extraction and comparison with ground truth, as well as for day-to-day applications.

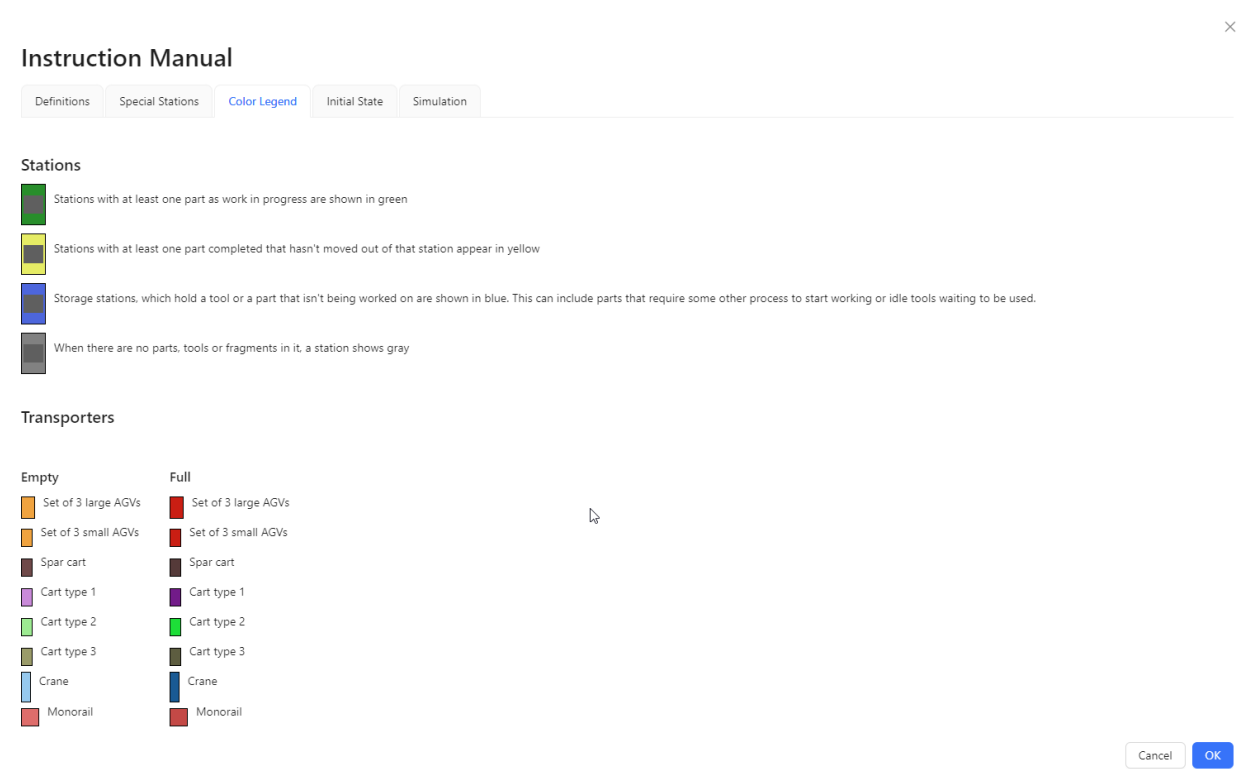


Figure 18 - Screenshot of instruction manual panel. The system has several components and a decent amount of complexity. This instruction panel was designed to provide users with an overview of the functionality and color codes of the simulation.

Talk about the initial tests, iterative design, frequent updates, and the final solution. The tool consisted on a SimPy simulation that had x, y, and x type of functions. In particular the algorithm for pushing elements out was xxx. The file structure was modular, with high priority in modularity and object-oriented design. Talk about how although initially the plan was to focus solely on the AGV management system and automating this system, it's such an interconnected system that it's necessary to model all movement in the factory to be able to predict AGV moves. Talk about building a cold start vs a hot start system, and how one had to be built after the other due to data constraints. Talk about the need for a user interface not only as a final product but also as a debugging tool. Talk about the design of the initial state property.

3.5 Validation and Reliability

Once the solution has been implemented, validation tests are necessary to assess the system's ability to represent the factory's state. Validation allows us to determine the reliability of the designed solution and, importantly, how it can be adjusted for more accurate results. Reliability is measured by comparing the simulated predicted state to the actual future state. Therefore, the simulation outcome is limited by the precision of the data collected about the simulation state. In this study, we categorize validation into two types: historical data validation and real-time validation. For our scheduling system to be trustworthy and fully automated, this accuracy should approach 100%, with simulated actions occurring within 20 minutes of actual actions. Unpredicted disturbances, such as machine breakdowns, should be the only sources of inaccuracy. Given the scarcity of data, we do not validate time accuracy, but rather whether a move occurs within a 10-hour window. This study refrains from predicting beyond 10 hours into the future due to high system volatility.

It's important to mention that, although the GUI is beneficial for end users, it provides significant value in the development process. One of the most powerful features of simulation is parallelization. Since events are triggered in parallel, having visibility of the system as a whole

is essential for debugging logic issues. Visualizing the simulation allows developers to follow along with its flow and detect unwanted behaviors and potential causes.

As mentioned in Chapter 3.3, stamps remain the sole method for determining the state of a process. Therefore, refining the simulation with historical data largely depends on stamps. For starters, validating the correct usage of buffer stations is not possible using stamps. Moreover, the stamps lack timed data indicating when a move actually occurs. Finally, the time window for validating historical results is limited. With these considerations in mind, we validated the simulation's results and iterated over new configurations, parameters, and algorithms.

Validating the efficacy of the simulation against historical results involved loading historical data at time t , running the simulation for 10 hours, computing historical data at time $t + 10$ hours, and finally constructing a confusion matrix to compare real and predicted moves. The 10-hour validation window aligns with operational constraints, ensuring reliable short-term scheduling. Predictions beyond 10 hours introduce increasing uncertainty due to stochastic factory events, requiring manual adjustments or additional constraints for longer-term scheduling. Since most production processes take significantly more than 10 hours to complete, a small fraction of the active parts in process are moved over a 10-hour window. Thus, relying simply on a measurement of accuracy is unreliable. Consider the case where 50 parts are in the system and only 3 of them actually move to another station. A model that always predicts zero moves would still result in a 94% accuracy. Hence, we rely on TPR as a proxy of simulation accuracy as its statistical significance is greater for our use case. Although a single simulation run appeared accurate, running the simulation for 30 t values and calculating the average accuracy and true positive rate (TPR) revealed issues with the processing times or logic of the stations that were subsequently fixed. Figure 19 highlights the significant improvement in metrics after adjusting those parameters. The decline in the last iterations was due to modifications in the measured timesteps. These additional tests were essential to ensure that the fixes did not overfit a single range of data.

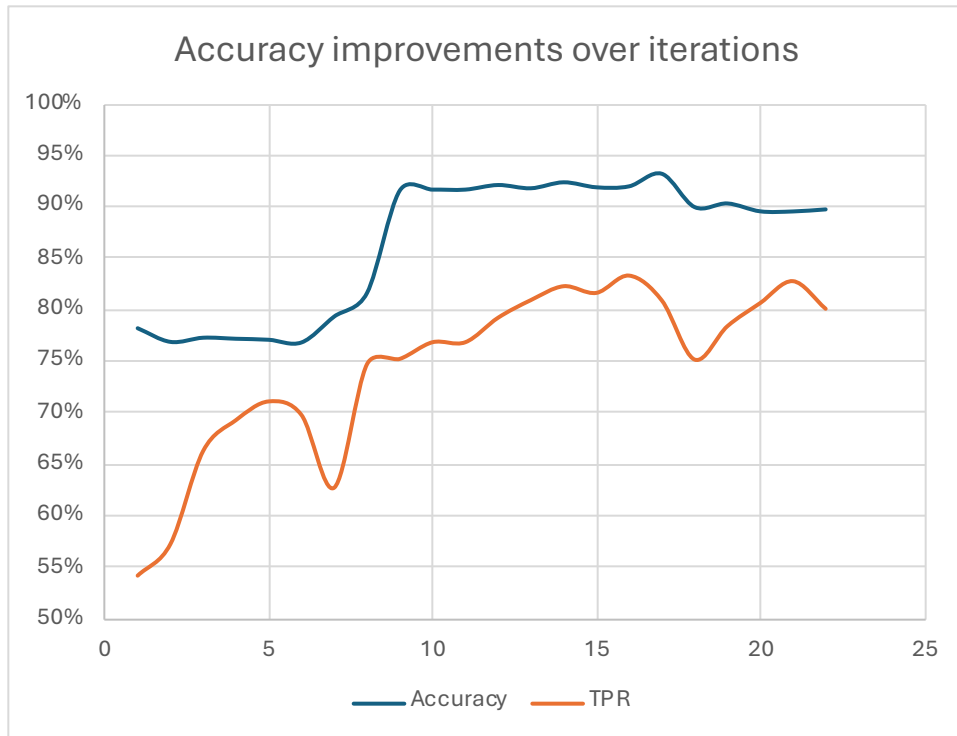


Figure 19 – Simulation accuracy improvements using historic data over 30 time periods. A total of 22 experiments were run, performing corrective actions and tuning the simulation to better approximate reality. More information on the actions performed for each new experiment can be found in Appendix 1.

Additional validation tests were conducted using real-time data at the end of this study.

Although the previously mentioned production stoppage led to inconsistencies in the data and the restart of production, it was possible to evaluate a week's worth of data on a daily basis. While quantitative information could not be gathered for this analysis, some potential issues with the simulation were identified and addressed. One example is the introduction of overflowing parts in the visual system. The problem that arose involved the initial state data indicating that a station contained more parts than was physically possible. This was mitigated by placing the most likely candidates into an array of overflowing parts and incorporating a visual representation of these on the interface, allowing users to restore those parts if necessary. Both the physical locations of parts and the daily AGV and crane movements were validated using real-time data.

Chapter 4: Results and Analysis

4.1 Findings

This study produced a series of qualitative and quantitative results. The interviews produced the business logic defined in Chapter 2, all of which would be implemented in the simulation software. Without these concepts, producing a simulation would have been impossible. These interviews also exposed the need for implementing a visualization system to provide an overview of the factory for stakeholders, along with a Graphic User Interface (GUI) that allows users to modify parameters. The simulation constructed produced a high accuracy predictive system, capable of up to 89% accuracy when using historical data.

Running the simulation over a time period of 40 days and obtaining the average daily moves, resulted in the following confusion matrix:

		Real	
		Moved	Didn't move
Predicted	Moved	36.0	15.1
	Didn't move	8.9	170.4

Figure 20 - Confusion matrix result of last simulation run. This table shows the simulated results as compared with ground truth from the gathered data.

From this matrix, we calculate an overall accuracy of 89.6% and a True Positive Rate (TPR), or Recall, of 80.2%. Moreover, a clear variability between the predicted end time and real end time was perceived. This variability is exemplified in Figure 21, which portrays, for every station, whether our prediction is going too fast or too slow. Correcting these values towards zero might seem like an obvious fix, but this results in overfitting, as running the corrected simulation on a different time period results in a drastically lower TPR. This is indicative of the extreme variability in station processing times.

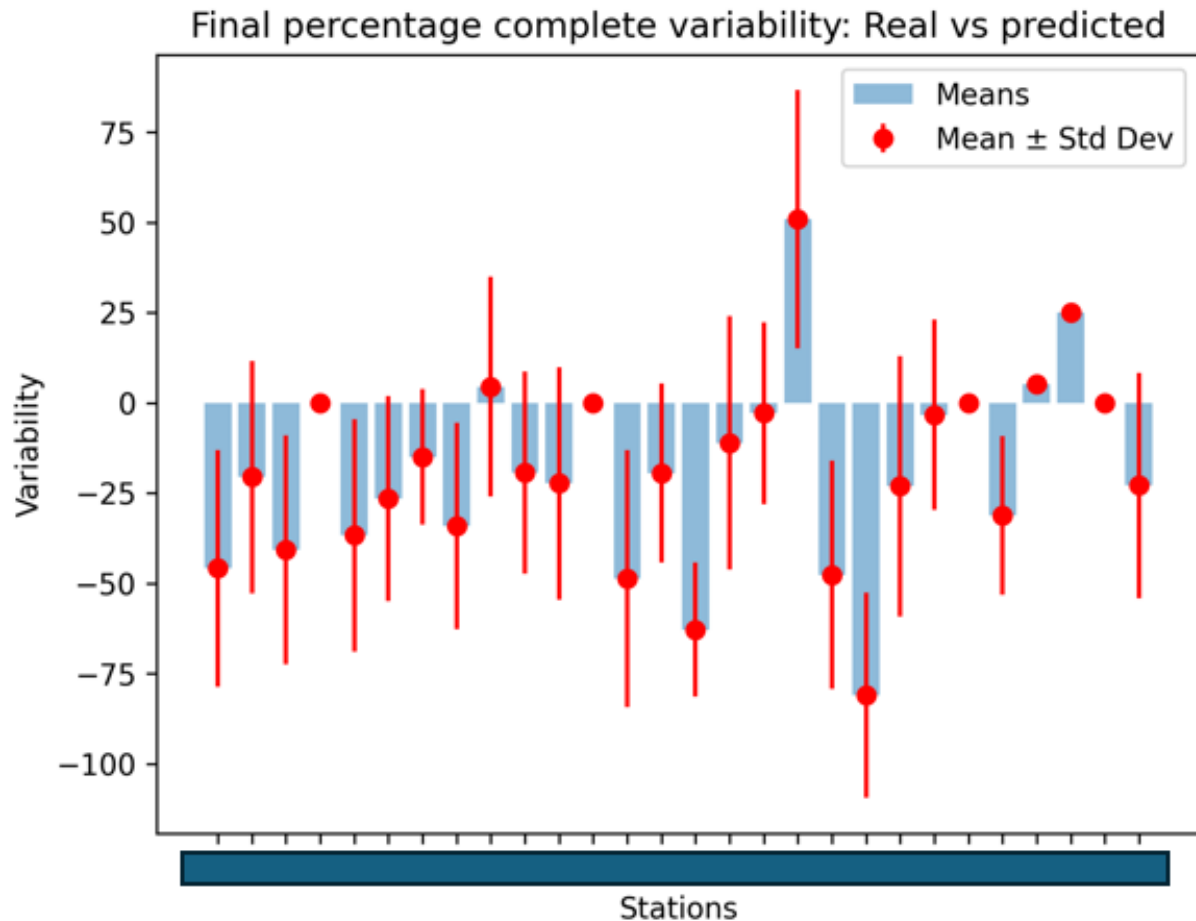


Figure 21 - Real vs predicted variability. This chart shows how our predictions perform against reality for every station. Essentially, the closer a bar is to zero, the better it's performing. At first glance, it seems that processing times for some stations should be corrected, yet modifying those values drastically reduces accuracy. The maximum accuracy was obtained with the variability shown in this figure.

This project culminated with a demonstration of the simulation as a packaged Electron application. The simulation was run using live data. The full feature set was demonstrated, including the simulation controls, instructions, and other interactive components. Stakeholder feedback indicated a successful delivery of a useful product. Although the initial assumption was that a scheduling system should be fully connected with the internal systems by publishing moves at certain times, the number of false positive and false negative moves make automating this system potentially troublesome.

In order to further understand the differences between the simulation and the real life state, the simulation was run for multiple days. The hypothesis for this consists of comparing the TPR

of runs over longer periods, which should tend towards 100%. Figure 22 validates this hypothesis.

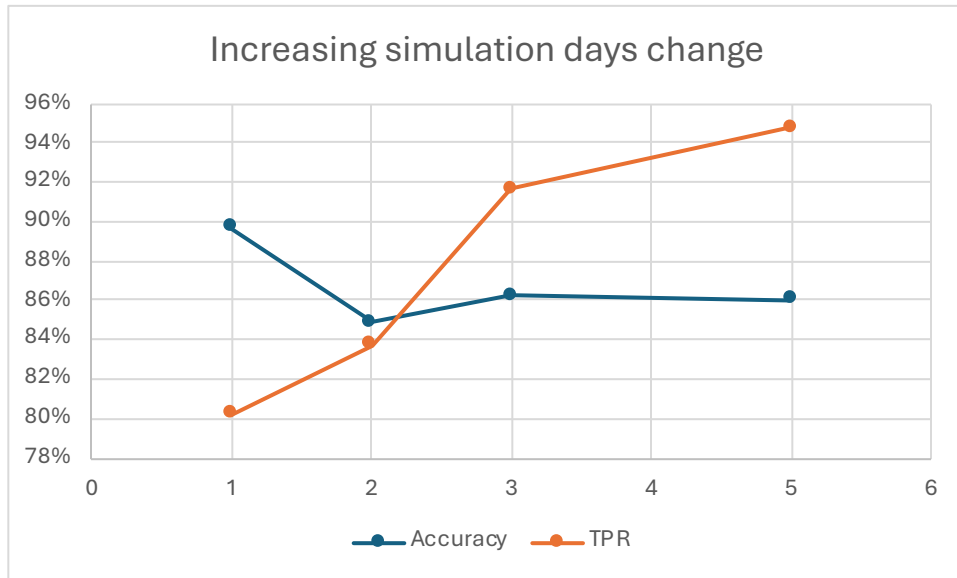


Figure 22 - Accuracy and TPR measured on simulations running over an increasing number of days. This chart indicates how increasing the number of days against which our simulation is running improves TPR, so if we run the simulation for 5 days and compare the locations of parts, there's a higher match than when we analyze only one day.

4.2 Interpretation of Results

If all the business rules were properly implemented and edge cases considered, the simulation should be as accurate as possible, yet we achieve only a 89.6% accuracy, an 80.2%, and an immeasurable time variability. It may appear that the simulation isn't properly set up, but at closer inspection, it's clear that factory variability is the main cause for concern. Variation of station production rates causes inaccuracies when predicting part movement. Nonetheless, the simulation's output can serve as a reasonable first guess for predicted movements. Stakeholders should be able to mitigate the 10% inaccuracy generated by the simulation in its current state.

Factory variability produces a constant simulation vs reality error of 15%-20% across different date ranges. This means that if, on average, 45 parts move in one 10-hour period, 36 of those moves are predicted to happen by the simulation, while 9 of them fail to be predicted. Worse yet, the resulting confusion matrix predicts that a total of 15 moves, on average, were simulated but didn't happen. This error rate comes primarily from oversimplified assumptions

of data, as we assume that stamp progress correlates linearly with time elapsed, yet this is rarely the case. Additionally, several stations contain large standard deviations, making accurate predictions more unlikely.

Unstandardized decision-making processes can also be a source of errors. As an example, operators may decide not to move a part to perform additional quality tests, while the simulation follows a strict push or pull algorithm. Additionally, overflowing parts contribute to the instability of the system, since running the 40-day simulation ignores the overflowing parts. Through the collected data, some parts would appear to be located at a certain cell, yet in reality would be located elsewhere, often for additional processing and inspection. The previous case is an example of how accurate location measurements can produce more reliable simulation results. Unavailable machines, machine breakdowns, and new factory configurations are also factors that contribute to inaccuracy. However, these can be potentially mitigated by teams utilizing the system who have context on the state of the factory and parts.

The solution discussed in this study provides the most value when paired with an industrial engineer who can fill in the gaps that the data is unable to complete. Moreover, the solution provides visibility of the factory state to a certain degree. It also enables future analyses of capacity and production rates. Improving data gathering of part, tool, and transporter locations can provide value in applications such as the one explored in this study. Furthermore, tracking part progress in a more granular fashion can help improve accuracy in the system.

4.3 Business Implications

Stakeholders at the company are impacted by the outcome of this project in various forms. Management now has access to a tool they can use for better-informed decision-making on factory decisions. Maintenance engineers could use the tool to schedule the maintenance of certain systems. Industrial engineers and operators can plan daily schedules according to the simulation. Most importantly, serves as a prediction and planning tool for improving factory efficiency.

This project lays the foundations for future developments at the company. Running efficiency analyses on top of the DES system that was developed can provide insights into whether the production rate can be successfully attained and what potential issues may arise. Analyzing factory modifications such as new cells for a station (increased capacity), a new set of AGVs, or the introduction of new machinery can be explored from a holistic standpoint, which considers the whole system as one. Connecting the simulation with the AGV vehicle management system (VMS) is also essential to reaching full automation. Although the current state of the simulation would be problematic when connected directly to the VMS, it lays the groundwork for potentially implementing automatic AGV deployment.

4.4 Technical Implications and Improvements

This study has demonstrated the impact of using visual representations of factory floors and processes to aid engineering teams. Through a well-distributed architecture, products with a robust backend and an easy-to-use frontend can provide value to stakeholders while also opening a communication channel for technical teams to connect and interact with the simulation. External technical teams can harness the exposed endpoints of the application to link real-time data with their own systems and perform their own automation. Additionally, a wide variety of simulations can be conducted to optimize certain processes. For instance, if a team of engineers wants to advocate for the introduction of a new machine in their process, they can run specialized simulations that connect directly with our system to obtain usable results and metrics to support their claims. This paper clearly demonstrates the necessity and advantages of DES for factory automation and optimization. Furthermore, the same system can be replicated in any factory where transportation scheduling is a necessity.

4.5 Limitations of the Study

First, validation of historic results depends on the same data that's being used to run the simulation, so if that data is inaccurate, the simulation results will be inaccurate. Second, many stations and flows were simplified for design purposes. The simulation assumes raw material

supply is always available, but this may not necessarily be the case. Similarly, the last portion of the flow consists of inventory storage, an additional rotation, a final inspection, and delivery. This additional rotation takes up production space in the rotation station which is used for other processes. This could potentially cause a bottleneck that's not currently modeled in our system. The final delivery step also takes over space in the hallway where parts are supposed to move through. This poses a potential risk of traffic jams. Conversely, using the simulation can help industrial engineers find time windows to perform the necessary moves to complete these steps without disturbing production.

The stamp structure poses a significant challenge for this study. Stamps give us a rough estimate of where parts are and how far along in their completion, yet more accurate measurements could be gathered by other means, such as cell-specific sensors or RFID sensors attached to parts. First, the stamps are operator-dependent, making them subject to human error. Second, stamps are only partially accurate as location predictors. Recording parts moving into a buffer station would require a sequence-agnostic system for geo-location.

Chapter 5: Conclusion

5.1 Key Contributions

Discrete event simulation has various applications in terms of identifying efficiencies, optimization, and predictions in a factory setting. Machine learning models often require large amounts of data to be able to accurately predict results. In scenarios where few historical data are available, DES is a powerful tool when several interacting parts are present, and understanding interactions is important. When using DES as a predictor, stochastic values for processing times, movement speed, and machine breakdowns are unnecessary since that stochasticity is unpredictable. Instead, we use averages and, on occasion, ideal values to understand where and when things potentially could move. Conversely, stochasticity can be introduced into the same simulation when analyzing machine rate improvements, throughputs, and potential issues with machine breakdowns. A well thought out and developed DES model can provide several advantages for industrial planning and execution.

In parallel, this paper explores the importance of visualization, human-machine interaction, and user experience as tools for understanding the state of a manufacturing plant. Developing internal tools for visualizing flows and deploying events that connect with external services is a complicated task that provides a lot of value. Providing engineers with a clear, holistic understanding of the parts moving through the factory automates a significant portion of the monotonous activities they perform on a regular basis.

Finally, the importance of accurate data gathering is fundamental to the growth and success of a manufacturing site. Initial data gather setup systems enable a factory to measure accurate data as soon as production begins and can even expose potential risks and opportunities in the testing phase. It's not necessary for a factory to have years of historical data to make progress, but it is imperative that they can obtain accurate measurements of current data, particularly where everything is, how far along in the process parts are, and what general policy is followed for moving parts along the factory. If this system is set up correctly, developing predictions becomes trivial.

5.2 Recommendations and Future Work

Developing this solution presented several opportunities for the company to improve its processes, as well as methods for expanding the final solution for other applications. As mentioned previously, the most important recommendation is the correct implementation of measuring devices to determine the position of parts in the factory. Alternatively, the application's modularity allows for several expansions to be implemented, including AGV policy optimization, user experience enhancements, factory section analysis, and capacity analysis. The simulation can also be further developed in certain aspects in order to improve accuracy and modularity.

The factory has previously implemented RFID systems that could serve as a viable solution. The main challenge with implementing RFID tags attached to parts is the high temperatures reached when parts go into the Crucible station, but aside from that, RFID would be an ideal solution to measuring where parts are in the factory. Paying attention to accurate data gathering, as well as cleaning and maintaining a database system, are fundamental for the

successful growth of the factory. Systems like the one developed in this paper are highly dependent on accurate and clean data, so increasing efforts on this front can potentially result in simplifying the development process for other internal tools.

Having a complete simulated flow of the factory enables the development of an optimal AGV selection algorithm. As mentioned in Chapter 3, there are several possibilities for AGV selection algorithms and for AGV idle policies. Running the simulation over several randomized configurations and measuring certain metrics such as throughput, AGV movement efficiency, and part wait time can provide optimal strategies for both AGV selection and idle policy selection. In a similar fashion, simulations can be run over a large number of randomized scenarios to predict what a modification in capacity or efficiency of a given station can result in. These results can be used as leverage by teams to convince management of the estimated impact that an investment in a given station can have.

The final delivered product also has several user experience improvements that can make it even more valuable. A feature that was discussed near the end of the development was the implementation of a capacity management section in the front-end. What this would look like is a UX component that allows users to add capacity and modify production rates to generate a new simulation. Some visual enhancements, like having the AGVs move smoothly around the screen instead of in time-jumps, would also result in a more seamless user experience. Finally, enabling users to close off a section of the factory and implement machine breakdowns can allow engineers to use the simulation to represent the factory state more accurately.

As mentioned at the end of Chapter 4, several simplifying assumptions make the simulation slightly inaccurate. Implementing the complete process flow from material sourcing to final delivery can be a way in which the application can be further improved. Additionally, harnessing the power of the machinery that already contains progress-measuring capabilities can be a way to obtain more accurate results when parts enter those stations. This would require connecting our system with another internal system not currently available to the platform and then combining the data from these two data sources, but it would possibly grant more granular details on the state of the factory.

Appendix

Appendix 1 – Accuracy and TPR improvements from modifying parameters

Iteration	Accuracy	TPR	Modification
1	78%	54%	
2	77%	57%	
3	77%	66%	In this iteration I validated moves of similar Gamma part names. So I no longer consider the specific Atlas tool, but the gamma part number to consider the move complete
4	77%	69%	
5	77%	71%	Fixed issue in code
6	77%	70%	
7	79%	63%	
8	82%	75%	This is NOT the same as the others. This test marked which part numbers did move, independently of their destination
9	91%	75%	Same as the previous one, in this test we looked at only the moves that happened regardless of the destination, but additionally we're ignoring the work orders that remained static in reality. If progress didn't change we ignore it. Those work orders generate a lot of noise
10	92%	77%	Similar to 9, but after fixing an issue to have gamma parts use the largest percentage in the set
11	92%	77%	
12	92%	79%	These results come from sending a single item to the Crucible Station when it's ready instead of waiting for four. We found a lot of the issues came from that. The simulation should wait for 4, but sometimes in reality they would go through with fewer
13	92%	81%	Capacity change for some station, it should allow for more parts for storage
14	92%	82.37%	Fix data issue
15	92%	81.723%	In this iteration we split moves into internal moves. That means if the flow is A-B-C, and we see a move from A-C, we split it into two moves: A-B and B-C. This is more accurate and closer to reality. It's interesting to note that the TPR and accuracy remained the same even in this closer to reality model
16	92%	83.404%	
17	93%	80.982%	This improvement comes from tweaking processing time

			This case went down in accuracy and TPR because we did an important systemic change: We added a priority list so workers work on parts in a particular order. Additionally, we removed all non-started work orders from the initial state. The priority list will be sorted by first created parts in
18	90%	75.234%	this case
19	90%	78.480%	We used a sorting algorithm by progress instead of by created_at date
20	89%	80.728%	Capacity changes
21	89%	82.863%	
22	90%	80.175%	Use march to august

Bibliography

1. GlobeNewswire. (2024, November 4). *Industry 4.0 market size to hit US\$ 862.0 billion by 2032: Rising demand for automation & smart manufacturing drives market growth*. S&S Insider. <https://www.x.com/news-release/2024/11/04/2974252/0/en/Industry-4-0-Market-Size-to-Hit-US-862-0-Billion-by-2032-Rising-Demand-for-Automation-Smart-Manufacturing-Drives-Market-Growth-Research-by-S-S-Insider.html>
2. SAE International. (2021, May 3). *SAE levels of driving automation™ refined for clarity and international audience*. <https://www.sae.org/blog/sae-j3016-update>

3. United Robotics Group. (n.d.). *Embracing the next generation of manufacturing automation*. <https://unitedrobotics.group/en/blog/article/embracing-the-next-generation-of-manufacturing-automation>
4. National Center for Biotechnology Information. (n.d.). *Article on manufacturing and automation*. PubMed Central.
<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9805798/#Sec2>
5. Martin, R. C. (2008). *Clean code: A handbook of agile software craftsmanship*. Prentice Hall.
6. Royce, W. W. (1970). *Managing the development of large software systems*. Proceedings of IEEE WESCON.
7. Beck, K., Beedle, M., van Bennekum, A., et al. (2001). *Manifesto for agile software development*. <https://agilemanifesto.org/>
8. Schwaber, K., & Beedle, M. (2001). *Agile software development with Scrum*. Prentice Hall.